平账魔法别想逃：源生整账与真实续行

Book-Balancing Magic Has Nowhere to Hide: Source-Native Accounting and Actual Continuation

高健（Jian Gao）

独立研究者 · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

源生整账法 · Source-Native Whole-Ledger Accounting

有差必有账，有结必有据。

账不只会记仇。它会自己生成下一笔，付钱，回答，再翻页。

摘要

我们构造一条由同一实际来源兑现的整账续行链：完整未来模型中的原残差生成下一请求的语法与责任行，源程序实际执行并严格支付同一债，完成事件同时交付回答、回执、完整账写入及真实后继。固定源及合法编译合同后，裁剪事件、借用异源回执、隐藏责任、等额迁账和以零预算冒充结清的具名候选均不能构造所需合法对象。构造对原源工厂、初始材料、配置及每个自然数编号、在运行拍上共尾的原准入点成立，保留全部允许作用词与站位的未来读数；原语法、已付状态、完整执行迹和责任归属一并进入同一次下一写入。数值效应为零也不删除原语法费用；新责任由自己的实际来源生成，不补充旧债预算。源生整账以完整事件库存与完整开放责任纤维为对象，有限处理行通过确切余项承担整账，拆并保留每个父项、子项、发生重数和原类型收据。对固定原债，有限不补充历史夹住一次严格付款，使付款宏步关系良基，无须总闭合律；另由逐当前发射准确结清或付款宏步的总律构造有限结清历史。最终结清同时要求原终回执和实际后继中同债纤维为空。原目标六展示真实修复、付款与结清；原整数消费者不能由全部有限复观察经任何函数恢复；在列明的右半零点回执输入下，有限窗口、逆残差、两次均值及余链的四份旧执行迹继续进入纠正程序、真实新库存与全部后续读口。整账因此既约束责任怎样合法结束，也生成它怎样继续工作。

关键词：源生整账；完整事件库存；语法付款；责任谱系；实际回答；完整未来模型；类型化结清

Abstract

We construct a whole-ledger continuation chain fulfilled by one actual source: an original residual in the complete future model generates the syntax and responsibility row of the next request; the source program executes and strictly pays the same debt; and its completion event delivers an answer, a receipt, a complete ledger write, and the actual successor. With the source and lawful compilation contract fixed, the named candidates based on event trimming, receipts from another source, hidden responsibilities, equal-budget migration, and zero-budget pseudo-settlement cannot construct the required lawful objects. The construction holds for the original source factory, initial material, configuration, and every actual cofinal ordinal, retaining future readings at every allowed action word and site. Original syntax, paid state, the full execution trace, and responsibility ownership enter the same next write. Zero numerical effect does not erase syntax payment, and a new responsibility receives its own source rather than refilling the old debt. Complete event inventories and complete open-responsibility fibres are the accounting objects; finite explicit rows answer the whole ledger through exact remainders, while split and merge preserve every parent, child, occurrence multiplicity, and original typed receipt. For a fixed original debt, finite non-refilling histories enclosing one strict payment make the payment macro-step relation well founded without a total closure law. A separate total law emitting exact settlement or an actual paid macro-step at each current constructs a finite settlement history. Final settlement requires both the original terminal receipt and an empty same-debt fibre in the actual successor. Target six exhibits actual repair, payment, and settlement; an original integer consumer cannot be recovered by any function from the entire family of finite complex observations; and, conditional on the specified right-half-plane zero receipt, four traces of a finite window, inverse residual, two means, and a cochain persist into a correction program, its actual new inventory, and every later reading. The ledger both governs how responsibility ends and generates how it continues to work.

Keywords: source-native whole ledger; complete event inventory; syntax payment; responsibility lineage; actual answer; complete future model; typed settlement

目录

摘要 1

Abstract 2

1 余额是零。结清回执呢? 4

2 整本账，比这一屏大 5

3 责任不靠换页消失 6

4 同一个零，可以藏着两本不同的账 9

5 可以迁账，不能借迁账换一条命 10

6 这次不是迁账：从 $2+4$ 到 $3+3$ 12

7 别拿终值打折：语法亲自付账 13

8 新账接管旧账，也接管下一问 15

9 原账自己开口：请求、付款、回答与下一次发生 18

10 真正付过一次，迁来迁去也补不回来 24

11 两把钥匙，才开结清的门 25

12 换一种语言，也得把旧账带来 27

13 整账也会长出新的数学效果 28

14 长计算不准甩掉自己的账 32

15 与原有方法的准确接缝 39

16 账不只记仇，它真的会付钱 40

附录 A 共同定理与本文的独立证明 41

附录 B 合同与计量单位 42

代码与数据可得性 42

参考文献 42

1 余额是零。结清回执呢?

把一本账的末行写成零，并不困难。困难的是解释：零掉的究竟是债，还是那一栏。

设有人报告“已经完成”。追问尚未履行的承诺，对方打开新界面：没有这一项。追问原承担者，对方递来迁移单。再追问接收方，对方展示合并总表：借贷相抵，余额为零。转移可以合法，数对抵消也可以正确。它们仍须回答同一个问题：原责任究竟接受了哪次处置?

源生整账把这个问题放回实际发生。源先行固定事件族、编译律、完整责任账和读口；同一次发生生成有限处理片段、每行去向与原后继。所谓“不可伪造”在本文取明确的类型语义：固定这份源与合同后，对具名非法候选构造从其所需资格类型到空类型的函数。这一定义将在裁剪、隐藏、拆并、等额付款及假结清中分别兑现。

但一本只会揭穿谎话的账，还少了一项最有用的本领：**它能否用自己已经生成的材料，亲自做下一件事?** 本文给出的答案是肯定的。完整未来残差进入原根，长成下一请求的原语法；语法执行支付同一数学责任；实际完成交出回答和原回执；原整账把全部旧项、这次新项与已付材料交给同一个真实后继。付款不是数值的别名，回答也不是贴在账上的奖状。

主定理 1.1（原账生成请求、付款、回答与续行）。固定任意原源工厂 $\mathcal{F}$、完整初始帧 I、程序 cfg 及其语言同一性 $\mathsf{cfg.LowVar} = X$。原运行自产一列准入停点，按 $n \in \mathbb{N}$ 编号，并在实际运行拍上共尾。对每个这样的准入点，由其原生出生程序给出的下一完整包与原根当前态满足以下合同。

1. 原文字残差与已付关系边界在每个环境中的语义相同；其关系同时保留在全部允许作用词及站位的完整未来模型中。整份材料、执行迹及责任归属沿真实当前态等式运输，下一请求由该材料生成。
2. 旧责任行与出生行均从这一原写入的完整账去向读取。下一请求的原语法产生正的剩余 credit；其实际付款分支给出同债唯一目标、严格预算扣减、不补充预算及同一次整账去向。
3. 原后缀执行完成该请求，完成值等于原材料的效应。实际结算动作由原库存目标编译为责任准入；其依赖消元同时返回原回答、回执、旧／新行及规范后继。
4. 第 14 节对每份列明的原解析输入给出有限窗口与逆残差的条件构造；四份旧迹、下一窗口库存和纠正迹进入这条实际续行，纠正请求、真实出生包及全部后续观察消费同次写入。

证明安排。第 7–8 节建立语法、责任出生、有类型库存及同一运行的完整共尾接口。第 9 节完整证明 (1)–(3)，第 14 节证明 (4) 的长计算条件合同。这里的“下一”每次都由原编译器生成，没有从未来商类中选一个新语法代表的步骤。最终结清另用原终回执与后继无同债的双条件定义；第 10–11 节分别证明付款良基性与消费总发射律的有限结清。

一般残差恒等式、出生付款及完成后缀对任意原工厂的实际材料成立。第 9.2 节还从任意 $\mathcal{F}$ 源内构造完整配对读口的共同查询工厂 $\widehat{\mathcal{F}}$，让实际查询直接读出同一当前态的旧值／效应整对。这是可生成的追加读口；一般链不以读口安装为额外假设。

目标六与原整数消费者分别给出两条独立实例。原目标六把 $2+4$ 修复为 $3+3$，同债预算 $1 \to 0$，再交出独立终账与无同债后继。另一原算术链上的同一非零整数类，在全部有限阶段的复观察下归零，而原整数消费者读出非零：任何函数都不能从整族观察恢复这个消费者。长计算部分另给条件应用：对每份列明输入，旧写入的残差成为下一纠正程序的输入，窗口读数与整份计算历史一起保留。

共同过程同一性、表示和源执行结果具名引用已发表的过程核心第二版 [6]。本文独立承担完整事件准入、逐行整账、责任拆并、付款与结清的区分，以及原未来模型到真实请求、实际付款、完成回答和长计算续行的证明。定理位置与分工见附录 A；详细编译关系与逐条证明见证明补充。

2 整本账，比这一屏大

2.1 先给责任身份，再谈读数

定义 2.1（完整账与源发生）。一个责任世界给出支承位置类型 S、责任类型 R，以及依赖类型 $\mathrm{Open}(s,r)$。$w:\mathrm{Open}(s,r)$ 是该责任在该位置实际开放的见证。位置 s 上的完整活账为

$$\mathcal{L}(s):=\sum_{r:R}\mathrm{Open}(s,r). \tag{1}$$

账项 $e=(r,w)$ 的语义主张 $c(e)$ 和自然数预算 $b(e)$ 均从 w 读取；位置另有锚点、发生关联及谱系键。两个见证支持同一预算读值，不表示它们是同一账项。“完整”在此不是一张表的美誉：$\mathcal{L}(s)$ 就是整个依赖纤维，没有让消费者勾选若干责任的字段。

一个源固定当前态类型 C、初始态、逐态事件族 $E(x)$、事件到支承位置的映射，以及两个相容的编译器。结构编译器把 $\varepsilon:E(x)$ 编译为原生写入、关系写入、续接运输、重定向或本地终账之一；前四者的后继由各自载荷给出，终账的本地后继为无。整账编译器从同一 ε 生成完整账的演化与生成式处理片段。随后安装的发射器 $g(x):E(x)$ 还须满足：编译器给出的目标发生，等于该发射器在目标态发出的发生。不能在箭头中间换一枚事件。

源也先行固定读口。对读口 p 和事件 ε，其结果类型是

$$\mathrm{Read}(p,\varepsilon)=\left(\sum_{a:\mathrm{Active}(p,\varepsilon)}\mathrm{Payload}(p,\varepsilon,a)\right)\sqcup\mathrm{Inactive}(p,\varepsilon). \tag{2}$$

分类、激活见证与有效载荷都是源的依赖输出。“未激活”因而也有自己的准确见证，而不是随手放入的空值。一个读口安装由单射嵌入和整项结果的交换等式给出；它保留的不只是数值，还包括分支及分支所携带的见证。

这立即约束后续消费者：同一完整源、同一事件、同一已安装读口，不能在两个安装路径上读出不同的依赖结果。证明先比较源分类；激活分支由和类型的单射性得到同一激活见证，再沿该等式运输有效载荷；未激活分支同理。安装交换式将两条路径都还原到这个原结果。另一方面，一个布尔构造确实让两个不同的完整源拥有相同的较低层根读值而产生不同读口：忘掉读口律以后“看起来相同”，正好不够。[1]

2.2 终账，不能靠删掉下一页

定义 2.2（完整事件库存准入）。设 $\mathcal{S}_*$ 是待呈现的具体源，$\mathcal{S}$ 是其权威呈现。准入包含当前态及每个当前态的完整事件族的双向构造性呈现：正反映射复合均为恒等。呈现还须交换初始态、结构编译、结构分支、可选后继、支承位置、完整整账写回和实际生成片段。若源有共尾边界事件，则边界事件族、是否发射、每个前驱及落点同样双向保留。这里的双向性针对完整族，不针对本拍挑出的子集。

最后，准入带有独立的**终账穷尽条件**：若一个呈现事件编译为本地终账，则对应具体当前态的完整普通出边族为空。普通出边是“一个实际源事件，加上该事件编译所得的续接后继”；不是某个读口声称存在的箭头。把这个条件写进合同，才有资格把局部终账提升为完整库存上的终账权威。终账以后的跨根接续由另一项源先行固定的接续律产生；它与本地普通出边不是同一个类型。

定理 2.3（不裁剪的终账）。在定义 2.2 的准入下，每个具体源事件均能往返恢复。呈现中的忠实终账对应具体源编译器的真实终账，保留原整账及原生成片段，并排除对应具体当前态的一切普通出边。

证明。事件一个不少，终账旁边仍可能开着一扇门；证明的分量在后半。事件的往返恢复直接由两条逆等式给出。设呈现事件为终账，后继交换式使对应具体事件的后继也是无。逐一展开具体编译器的五个构造：四个续接构造

[1] 证明使用分类等式与安装交换式；布尔反例更换的是完整源，而非固定源下的结果选择。

的后继均为“有”，与“无”矛盾；余下的只能是原终账构造。整账和片段的交换式给出完整依赖对象的相等。最后使用终账穷尽条件，得到所有普通出边为空。事件双射本身只保证没有删事件；排除终账旁仍有出边，确实使用了最后一个条件。证毕。

反例 2.4（裁掉的不是噪声，是出路）。取只有一个当前态的具体事件族，内有两个不同构造：终账事件 t 与真实写入事件 w。w 的编译结果带一个实际后继。另取只保留终账事件的单点呈现。若它是完整呈现，正向映射必把 t, w 都送到唯一元素，反向复合恒等遂给出 $t = w$，与构造不同矛盾。即使只要求选中的 t 可以编译为终账，w 仍直接否定终账穷尽条件。这一反例的对象是混合事件源及其被裁薄的呈现。

在这里，删除责任和删除行动机会是同一类手法：先缩小全集，再在剩下的部分宣布万事大吉。定理 2.3 使这种手法无法通过类型检查。

2.3 发生身份不是状态标签

命题 2.5（登记发生与差异定位）。一个实际访问由当前态连同源生成的到达历史组成；历史从源初始态开始，逐步沿原编译器的后继等式推进。有共尾落点的历史还保留原边界事件及此前驱流。登记发生为

$$\mathsf{Occ} := \sum_{v:\mathsf{Visit}} E(\mathrm{current}(v)). \tag{3}$$

不同访问上的两次发生不能因当前态标签相同而被视为同一次。对固定的源生过程，把某个登记发生定位为该过程的某次访问与其原事件，就给出了正面的内部结构身份。因此，对这一登记发生语义，每个实际、可稳定指称的真实差异均可定位；它不可能同时声称“非冗余”与“没有任何内部结构身份”。

证明。同一个当前态可以被走到两次，能分开它们的只有到达历史。对依赖对的相等取第一投影即得访问相等，故不能消去不同的因果历史。源生过程的状态嵌入对完整根与访问单射；过程登记反解所得到的状态与原访问一致。用这个状态、访问和原发射事件构造结构身份即可。更抽象地，若实际差异被分类为冗余或具有结构身份，所谓外加的非冗余、无内部决定的差异在前一分支与非冗余性矛盾，在后一分支与无内部决定性矛盾。这里先有正面的定位构造，排除无账差异才是其推论。

这是“有差必有账”的第一个意思：看不懂的差异不能删，它在所声明的实际源中必须有一个可恢复的位置。第二个意思紧随其后：在这个位置上，谁的责任仍然开放。

3 责任不靠换页消失

3.1 原准入留下名字，处置留下回执

定义 3.1（准入、活责任与终档）。一项已准入责任保存原准入回执，并另有当前内容、剩余事项、承担者、受益方和适用范围。原回执记录源事件、源锚点、责任发生关联和谱系，以及该源有权产生该项责任的证明。需求、承诺、委托、任务、依赖或风险可以提供准入起点；起点还须接上源权限、接受或授权承担、以及履责管辖。单有需求，并不生成别人的义务。

活责任可以接受进展、维护、延期、范围调整和已接受的转移；这些处置必须各自携带原合同要求的见证。范围缩小需要相应管辖，转移需要接收证据。取代保留覆盖原义务的实际继任责任；承担者解除保留合法的目标责任或上层处置路径。二者都不是把旧责任无声删除。

终回执有五种有区别的合法形状：履行完成、获授权豁免、原责任被判无效、承担后果的放弃、被接受的不可实现残留。它们结束责任的根据不同，不能都重命名为一次数值付款。终档保存被结束的完整责任、源事件和终回执；重开从该终档出发，以新的实际准入为输入并保留原谱系，而不是另发一个“从未欠过”的身份。

定理 3.2（没有凭空新生，也没有静默消失）。在由上述原生边生成的任意有限生命周期历史的每条实际边上：一个此前不存在、随后活跃的新槽位必有实际准入回执；一个由非活跃变为活跃的已有位置，只能由准入或重开生成；一个活跃位置变成非活跃时，必须保存该位置上那项原责任的实际终回执。

证明。每条边只动一个位置，所以逐边查清谁能让位置亮起、谁能让它熄灭就够了。对单边先作穷尽分析。准入把一个带原回执的活责任追加到槽位序列；重开只作用于已存在的终档。其余边都带一个被操作的活位置：若所考察位置不同，它的值不变；若相同，进展、维护、延期、缩小、转移、取代和承担者解除仍产生活责任，只有终结构造把它换为携带原责任及终回执的终档。因此三个结论分别只能落在准入、准入或重开、终结的构造上。有限历史按首尾相接归纳，将首次改变该位置状态的实际边及其回执保留下来。证毕。[2]

一个具体的承诺准入把空状态变为一项剩余量为 3 的活责任；其被接受的转移把承担者改为继任者，槽位仍然活跃，谱系不变。另一个拒绝准入的世界有可提出的需求，但接受和授权承担的类型均为空，因此没有准入值：不是“未找到同意”，而是其指定准入类型被证明为空。

3.2 这一拍只改几行，不等于世界只剩几行

定义 3.3（整账演化与生成式片段）。对一次续接事件 $\varepsilon : s \to t$，完整账的演化有两个全域输出：

$$d_\varepsilon : \prod_{e:\mathcal{L}(s)} \sum_{f:\mathcal{L}(t)} \mathrm{Evolve}_\varepsilon(e,f), \qquad o_\varepsilon : \prod_{f:\mathcal{L}(t)} \sum_{e:\mathcal{L}(s)} \mathrm{Evolve}_\varepsilon(e,f). \tag{4}$$

前者回答每项旧责任的去向，后者回答每项目标责任的来源。二者不是预设的互逆双射；拆与并正需要保留多对一或一对多的关联。

式 (4) 是固定责任世界内的普通续接合同。责任出生与表达修订先由原源生成新的责任世界及其初始账，再对该世界的完整纤维使用这个合同；新责任的第一写入因而从已经含有出生行的初始账出发。原生准入仍须保存定义 3.1 的源发生。本地终账另给 $\prod_{e:\mathcal{L}(s)} \mathrm{Discharge}_\varepsilon(e)$：非空账需要逐项、以原账项为索引的结清回执，空账才可空泛成立。第 5.2 节的完整生命周期接口同时包含两种和类型：旧项给实际后裔族或终回执，新项给实际祖先或本次源准入。

编译器不是接收一张事后填好的式 (4)，而是接收源发生，生成有限显式行及其覆盖方式，再折叠为式 (4)。续接片段的覆盖方式有且只有以下三种。**原位余项**用于同一支承位置：查询命中时返回带“这正是所问账项”的等式的行索引，未命中就原项携带。**完整有限覆盖**给出源账和目标账到有限行号的全域映射，并证明每个行号真的返回被询问的项。**运输余项**允许跨位置的有限例外，但未命中的项必须交给同一个源生成的余项事件；余项事件的编译结果已经确定全部去向与来源。

终账另有两种生成方式：用有限终结行对完整责任纤维给出准确索引，或者使用由原终账源实际选中的一枚支承结清发生。后一种将同一个源编译的支承回执按原账项索引应用于整个纤维，无须假定账项有限；它并不接受一份事后补来的任意支承回执。

定理 3.4（有限呈现，整账回答）。每个上述续接片段均折叠为式 (4) 的完整演化；两种终账片段均折叠为完整逐项终结。所得结果与同一次源发生的整账编译结果相等。原位余项中未命中的账项是确切的原项携带，不能暗藏转移。完整有限覆盖不能把一个实际无限的责任纤维伪装成有限清单。运输余项则允许有限例外之外的完整纤维继续被处置。

证明。难处在没有被列出的那些项：它们没有行号，却必须有确定的去向。原位情形逐项分析可选索引：命中时沿所附账项等式运输该生成行，未命中时返回本项及恒等携带见证。未命中分支的处置构造不是转移。完整覆盖情形对任意旧项取指定去向索引，对任意新项取指定来源索引，分别沿覆盖等式取得生成行；这些等式使从有限行到每个完整纤维的映射满射，因而该种覆盖要求纤维本身有限。运输余项的命中分支同前，未命中分支直接调用源已生成

[2]共同底座见 [6] 命题 6.1。

的余项去向或来源。终账的有限行分支沿完整索引的等式取得每项终回执；支承结清分支则将准确被选中的支承回执应用于任意原账项。最后，片段的类型索引正是本次整账编译结果，折叠相等是编译合同的一部分，不能在折叠后另换一张总表。证毕。

一个具体付款根正采用这一覆盖方式。源生受门控主张的付款根，以一次实际严格推进和其准确目标上的整账终回执为输入。它的付款片段含 **零条显式行**，所有查询均选择余项；但余项事件编译为那次原推进的完整账演化，且折叠结果逐项等于原付款演化。原目标六的受门控付款即按此构造。这里的零是显式例外条数，不是责任条数，更不是“无需付款”。本例与第 6 节的原目标六行动链分别保留各自的付款对象。



图 1　完整事件库存与本拍处理片段是两种不同的完整性问题。上层的双向呈现不裁掉事件；下层的有限行加确切覆盖回答整个责任纤维。

3.3　问到哪一项，就回答哪一项

定理 3.5（逐项处置与同一后继）。固定一个已准入的源生根、一次登记访问及其整账发生。对该次完整活账中的每一项，原编译器均给出准确的目标项演化或该项的终回执。若是续接，目标项位于同一根实际发出的后继事件的完整账中；若已持有该项的生成式因果权限，后继权限由同一次实际演化导出。任何两项的“回答并继续”都读取同一实际后继当前态。

证明。要防的是目标项落进一张长得很像、却不是实际后继的账。展开原整账编译结果。四个续接构造各自给出目标发生和式 (4) 的去向。由本地后继等式的单射性，该目标态等于被询问的实际后继；由发射器交换式，目标发生也等于根在该态实际发出的发生。因此沿两个等式运输的目标项，不是另一张相似账中的候选。终账构造则直接调用完整终账演化的逐项回执。初始因果权限取自同一覆盖选择器选中的原生成行，库存中存着一行并不足够；续接权限由原权限及刚才的准确后继等式导出，不从目标侧重新准入。“回答并继续”的后继是原发生的编译读出，与询问哪一账项无关。证毕。

命题 3.6（真实处置迫使准确生成行出现）。若原完整账的本次处置是终结，或某项在原折叠演化中确实被转移，则可从这一处置事实生成该原账项的规范行权限；续接时的目标与依赖演化、终结时的回执，正是整账折叠所使用的结果，不需下游补交行号或库存成员证明。

证明。关键在原位余项：没命中时它只能携带，所以一次真实转移必然命中。终结分支使用有限终结的全覆盖选择器，或源生成的支承结清选择。续接分支检查片段类型：完整有限覆盖已经固定行号；运输余项无论命中显式例

外还是未命中，都分别从同一选择器生成例外权限或余项权限；原位余项若未命中，折叠结果必为携带构造，与假定的转移构造不同，因此必须命中。最后沿片段折叠等于原整账编译的等式运输，获得目标相等及依赖演化的异构相等。证毕。

每项已登记责任都获得准确回答，包括本拍清单没有列出的项。现在可以问：那块只显示总数的屏幕，到底隐去了什么。

4 同一个零，可以藏着两本不同的账

定义 4.1（相对于消费者的观察安全）。设 A 是待观察的完整状态类型，$q : A \to O$ 是观察者，$c : A \to D$ 是所关心的消费者；依赖处置可连同其索引打包到 D。称 q 对 c 安全，若

$$\mathrm{Safe}(q, c) :\Longleftrightarrow\ \forall x, y : A,\ q(x) = q(y) \Longrightarrow c(x) = c(y). \tag{5}$$

隐藏差异见证则是一对状态，携带 $q(x) = q(y)$ 及 $c(x) \neq c(y)$。安全从来不是观察者的一项绝对美德：同一读口可以足够比较余额，却不足以判断某项原责任是否获释。

定理 4.2（隐藏差异不能被认证为安全）。给定隐藏差异见证，$\mathrm{Safe}(q, c)$ 为空。对任何进一步压缩 $h : O \to O'$，同一对状态仍构成 $h \circ q$ 相对于 c 的隐藏差异。若另已给出实际分解 $c = \bar{c} \circ q$，则 q 对 c 安全。

证明。将见证的 $q(x) = q(y)$ 代入式 (5)，便与 $c(x) \neq c(y)$ 矛盾。进一步压缩只需对这个观察等式应用 h；消费者不等仍在。已有分解时，对 $q(x) = q(y)$ 应用 $\bar{c}$ 并使用两端分解等式即可。这里的安全定义是纤维内恒定性，不暗中加入构造解码器所需的选择；分解是足够条件，其一般逆向构造属于另行声明的经典选择适配。证毕。

实例 4.3（已经发生的隐藏准入）。回到第 3 节那条实际承诺准入边。薄观察者按一个始终不显示该责任的范围过滤槽位；它在空状态和准入后状态都返回空清单。完整当前责任观察者在准入前读到空槽位序列，准入后读到已准入的活责任；长度已经足以区分二者。原准入事件、不可见等式与这项完整责任状态差异共同生成重定向见证，保留薄观察所隐藏的实际变化。

因此，界面说“没有变化”时，可以有三种不同结论：它确实足以判断当前消费者；它已被一项隐藏差异反驳；或者尚未取得分类。第三种不是第一种的温和写法。整账法允许暂不判断，却不把缺证据包装成安全。

这条实际隐藏准入边一出现，消费者安全类型就是空的。同一判据还会在第 13 节的代数里再出场一次：全部有限阶段的复数化观察都归零，原整数消费者却明确读到非零。

4.4 原回执还会交出完整恢复或准确残差

命题 4.4（同原转移的四支数学处置）。原实际原生写入的回执生成自己的完整历史、关系和评价器。其源数学处置给出以下四支之一：完整线性恢复；一项实际关系的非零评价；原完成载体中的非零核项；或一个不在原评价映射像内的准确代表。最后一支对任意恢复函数都给出不能恢复该代表的证明。

证明。先从原编译器的实际写入分支消去回执，取得同一发生的生成元、种子、继续结构与评价器。设 E 为它在完整自由词模上的线性延拓。若一项实际关系被 E 读成非零，交出该关系与非零像。否则关系可靠，E 下降为完成载体上的 $\bar{E}$。其核非零时，交出一个非零核项及零评价；核为零且像完整时，原构造生成完整线性等价与其逆；剩下的覆盖失败支从非零余核类取代表 a，得到 $a \notin \mathrm{im}\,\bar{E}$。任取函数 f，若 $\bar{E}(f(a)) = a$，$f(a)$ 就给出 a 的原像，矛盾。各支的具体关系、核与余核见证明补充 A。原分类是非计算性的数学处置，原源程序的语法执行费仍使用自己的收据；原转移行、完整账和字面后继同时保留，没有把一个新残差读口冒充严格付款。证毕。

5 可以迁账，不能借迁账换一条命

5.1 移动的是承担关系，不是欠债的过去

定义 5.1（同一站位与同一债）。对源账项 e 和目标账项 f，同一站位要求锚点、发生关联、谱系、责任标签和语义主张全部相同。同一债只固定决定延续义务的谱系与语义主张：

$$e \sim_{\mathrm{debt}} f :\Longleftrightarrow\ \lambda(e) = \lambda(f) \text{ 且 } c(e) = c(f). \tag{6}$$

因而合法的承担者、锚点或局部时钟变化可以保留同一债。称目标项相对于完整旧账真正新生，须给出对每个旧项 e 的反证 $e \sim_{\mathrm{debt}} f \to \bot$；仅仅“它不等于某一个旧项”不够。

命题 5.2（续接不补发预算）。原整账行演化分为三种：原项携带保留同一支承位置和准确账项；维护保留同一站位并严格减少预算；转移携带真实转移回执，保留同债身份并满足目标预算不大于原预算。因此每条合法行演化都满足

$$e \sim_{\mathrm{debt}} f, \qquad b(f) \le b(e). \tag{7}$$

原项携带的预算相等；维护的预算严格小于；转移可以严格小于，也可以等于。

证明。对行演化的三个构造分别消去。原项携带的支承位置等式与账项异构相等把目标改写为原项，两结论均自反。维护直接读出谱系、主张等式及严格不等式，后者蕴含非增。转移读取其谱系、主张等式和不补充预算字段。证毕。

转移的等号值得单独留下。它不是失败的合法性证明；它是合法但尚未付款的情形。换一个承担者、目录、坐标或时钟，最多解释为什么责任现在出现在另一处，不能解释为什么它已经履行。改名回执同样保留原发生关联和谱系；尚未被接受的转移提议甚至不改变承担者。

5.2 拆开以后，每一片还得有来历

定义 5.3（逐项覆盖的拆并）。拆分回执固定一个拥有原责任发生的源事件、非空子项列表和整体覆盖见证；每个子项都有原父项到它的后裔关系、锚点运输、同谱系证明及局部履责保持。合并回执固定非空父项列表与目标项，并为每个父项保留祖先关联、锚点运输、同谱系证明及局部履责保持。整体覆盖不能替代其中任何一项。

在完整根账上，这些回执进一步由原事件和原整账演化索引。若两个不同目标项有同一个原来源，原重组编译器必须生成覆盖该来源纤维的拆分回执；列出的每个子项确在目标账内，目标账中属于该来源纤维的每项均被覆盖。每个子项的预算严格小于其父项。若两个不同旧项有同一个目标，必须生成同样精确的合并回执，逐一覆盖目标纤维中的原父项，不漏项，也不增添幽灵父项。若两项本来相同，编译器可以返回同一性分支，而不伪造拆并事件。

定理 5.4（拆并不能以总额代替责任覆盖）。在定义 5.3 的源固定重组合同中，非平凡拆分保留每个真实子项的来历与局部履责条件，非平凡合并保留每个真实父项的局部履责条件。对子项数或总额的相等，不能代替这些依赖回执。特别地，预算为零的父项没有合法的非平凡拆分。

证明。总额相等说明不了任何一项的来历；起作用的是覆盖条件与成员条件。对源固定编译器的分类作两支分析。同一性分支说明被比较的依赖账项相等，因而不能用于两个不同项。拆分分支利用“实际来源纤维全部覆盖”把任意真实子项送入回执的子项列表，再逐项读出后裔、谱系及局部履责见证；“列表各项均在实际目标账内”排除以虚构项凑覆盖。合并分支对实际父项使用对称的覆盖条件，保留各自履责见证。零预算时，任一真实子项都要求 $0 \le b(\mathrm{child}) < b(\mathrm{parent}) = 0$，矛盾。证毕。

严格下降是对子项逐一陈述的，不能偷换成“所有子项预算之和下降”。拆并的守恒对象首先是责任关联与履责覆盖，不是票数，也不是一项随手相加的总量。

反例 5.5（两行，仍然可以洗掉一行）。原弱重组构造的旧、新账项均由布尔值索引，因而两边各有两项；去向和来源却都恒取 `false`。每条被选择的成对转移都能给出 $0 \le 0$，两边条数也完全相同。但目标 `false` 与 `true` 具有同一原来源，旧项 `false` 与 `true` 又具有同一目标。完整分类若返回同一性，便要求两个布尔项相等；若返回拆分或合并，则必须提供该构造刻意留空的覆盖类型。两条路均矛盾。这是弱数据不能升级为完整重组认证的反例。

完整生命周期中的双向账也据此检查两端：旧项要么有非空、带覆盖的后裔族，要么有源拥有的终回执；新项要么有实际祖先，要么有本次源事件的真实准入。两端关联还须相容。消去这两类和类型，就分别得到“无静默旧项汇点”和“无免费新项来源”。这比清点前后各剩几张票更强：票可以拆，责任不能借机蒸发。

5.3 旧拆并与整本余项进入真正执行

定理 5.6（原重构证书随计算提升）。固定原权威根、从其实际编译器读取的完整原生程序及原登记输入。追加数学执行保留每枚旧责任的完整类型字段和原 split ／ merge 证书：子项、父项、source ownership、锚点运输、谱系、局部履责、全覆盖、无幽灵成员及逐 child 严格扣减，均进入同一实际追加写入。另给原源律生成的逐当前完整继续载荷程序时，一般继续分支的有限数学行加同源完整运输余项，折叠为原整账与数学行的 whole join；其真实 tick 直接消费这个证书、全部旧去向及原后继。

证明。用旧／数学两个不交的标签构造责任类型。旧责任嵌入 j 有读取左逆 q，逐字段满足 $qj = \mathrm{id}$：原准入、权限、假设和履责对象都被读取，而非只保留一个责任编号。旧完整去向与起源在 j 下运输；数学行只关联数学标签。因此旧端的重复来源或重复去向恰反映原编译器中的同一个重构问题，数学端不与它产生交叉纤维。

原分类为同一性时，沿左逆把依赖项相等提升。分类为拆分时，将原 children 列表逐项送到 j；列表成员、全来源纤维覆盖和无幽灵成员经 $qj = \mathrm{id}$ 双向保持，逐项运输后裔、锚点、谱系、局部履责与严格预算不等式。合并对全部原 parents 做同样运输。故目标没有收到一张另行提交的分类证书，收到的是原母证书的像。

给定原源律的逐当前完整继续载荷程序，对其 native、relation、continued transport 和 redirect 任一实际继续载荷，先消去原编译结果，得到该分支自己的整账演化、生成片段和后继。完整 join 的旧去向是原去向，数学去向是本次真实执行步的去向；来源同理。有限片段只显式保存数学行，未命中的所有旧项交给刚消去的那枚原余项事件。对完整旧／数学 Sigma 分支逐项展开 fold，便等于 join 的完整去向与来源。原重构证明按上段提升到该 fold；同一次 tick 的编译等式再把其证书、去向及后继认回实际编译结果。证毕。

完整原生有限安装还有直接的位置读法：若旧行数为 m，新库存为 $m + 1$ 行，旧位置沿原末位扩张逐项进入，数学行占新末位。重复主张和相同载荷没有去重。旧同一性证书保证旧两端单射，旧／数学标签分离保证新行与它们不碰撞；一般 split ／ merge 情形则消费上面的原重构证书。完整余项形式不要求旧责任纤维有限，覆盖责任也不由有限行数代付。

拆并至此不是一枚静态许可证：原母源已经支付的全部 Type 收据跟着原程序进入实际下一拍。第 7 节将用这条全账方块，让无旧承担者的语法责任在保存旧账的同时亲自付款。

6 这次不是迁账：从 $2+4$ 到 $3+3$

6.1 让原行动发生，再读取它的答案

原目标六过程从 $2+4=6$ 出发。右侧的 4 不是素数，其实际最小因子读值为 2。原生产编译器的有限因子修复库存包含这次修复的结构键，因此它的“已完成”分支被 4 非素数排除，“无可用修复”分支被库存成员见证排除；留下的是原编译器已经生成的行动分支。

行动按原修复通道把左侧改为 $2+(2-1)=3$，并由总目标 6 固定右侧为 3。因此得到的不是一个提交给编译器的期望答案，而是其实际目标：

$$(2,4)\xrightarrow{\text{原因子二修复}}(3,3),\qquad 3+3=6. \tag{8}$$

这里的行动回执只携带原步骤、原步骤生成的目标，以及它确为原编译器输出的等式；其中没有“素数对已存在”“占据非零”或“已经结清”的自由字段。

接下来才读取目标。原目标分类器检验两侧素性：若仍进入行动分支，就与 3 和 3 均为素数时其分类等式矛盾。因此目标分类器发出实际终类读值。由这个读值取出有效素数对纤维，再由该纤维构造原实际性结算，得到原占据读值非零。原行动与终类读取是两个依赖步骤：后者必须读前者的准确目标，不能向前者塞入一个预先准备好的结论。

6.2 同一行动进入责任账

定义 6.1（目标六行动债）。取开放态 u 与已付态 v，预算为 $b(u)=1, b(v)=0$。该债律的付款步骤由上述原行动生成，终结见证由上述目标终类读取生成。把此债律接在已发出的原算术发生上：扩展活账保留全部旧项，并增加这一项行动债。发生索引的激活源发出原付款分支；直接编译器把这个分支编译成整个扩展账的续接演化。

定理 6.2（原目标六的真实付款链）。原目标六行动给出同债预算 $1\to 0$ 的实际严格付款、准确已付目标上的独立整账终回执，以及由该终回执驱动的债务不再激活的原算术后继。付款、终账和接续分别来自对应的原编译器，不由“预算已经为零”相互代造。

证明。付款、终账、接续三步各有自己的编译器，最容易省掉的是最后一步：终账之后世界还在走，要看它的后继里这笔债是否不再激活。先由式 (8) 的实际行动得到行动债的生成步骤；其起点、目标与预算读取分别化为 $u, v, 1, 0$，所以严格不等式是 $0<1$。扩展账演化逐项搬运旧责任，并把这项债的实际步骤编译到同一债的目标项；源发出的分支等式把它固定为本次付款，而非同型的另一次候选。

随后，目标终类读取产生 v 上的支承终结见证。整账终结编译器以它为输入，为已付支承上的完整活账逐项给出终回执。原激活源发出付款分支。独立的两阶段正向根把“开放阶段的原付款”和“已付阶段的准确终回执”分别安装为写入发生与终账发生。已付阶段的事件族只有终账构造，其普通出边族为空，故通过完整事件库存的终账准入。

最后，源先行固定的终账接续律作用于这个已登记的终账发生，生成原算术下一态，其扩展支承的债状态为“未激活”。该完整目标账只有原账项的继承分支；新增债分支没有开放见证。任取一个目标项，它属于旧主张分支，不能等于被结清债的主张分支，故实际后继中的同债纤维为空。原账没有被删除，指定行动债确实不再开放。证毕。

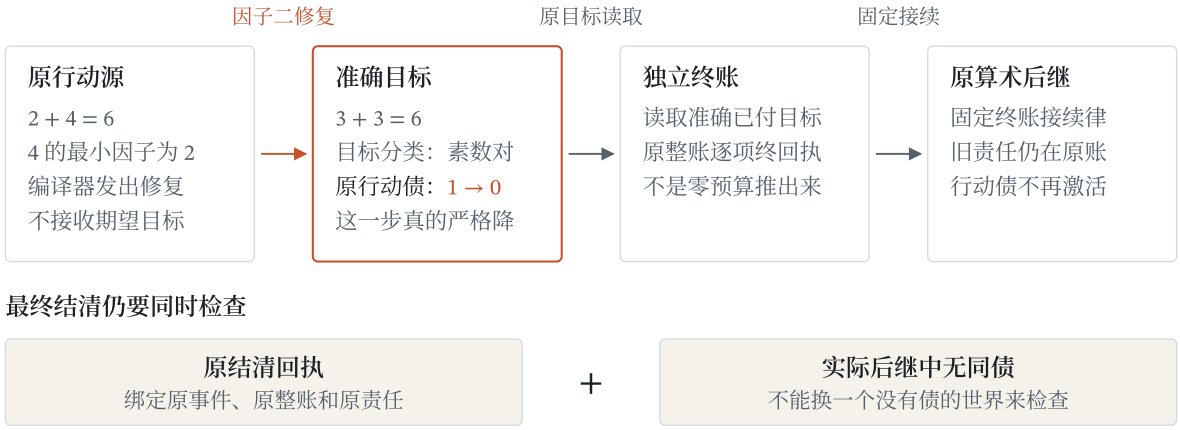


图 2　原目标六链：原行动产生准确目标，目标读取支持独立终账；零预算不是两者之间的捷径。结清后的原算术后继保留旧账，但不再激活该行动债。

这四步各做各的事。$2+4 \to 3+3$ 支付一个具体行动；$1 \to 0$ 记录它在指定责任尺度上的严格推进；独立终账给出该支承的逐项终回执；后继检查确认这项行动债没有换个位置继续欠着。把任何一步删去，余下的数字仍可能漂亮，结算合同却不再完整。

7　别拿终值打折：语法亲自付账

7.1　没有旧承担者，程序仍有自己的责任

固定一个已准入的完整根、其原发生 o，以及从 o 读取的环境—表达式对 (ρ, e)。表达式的原剩余计数记为 $r(e)$；每次原执行步消去一个语法节点，恰使它减一。这个单位贯穿本节：线性、双线性等原语所占节点按原语法计数，其内部算术或位运算仍由相应原语合同计量。

定义 7.1（有偿状态与关系收据）。有偿状态是 $x = (p, t)$，其中 $t : e \longrightarrow p$ 是在原环境 ρ 下已经发生的完整执行迹。它保留原表达式、当前待执行式、环境、每个中间步及来源；预算为 $r(p)$。原执行器在非常量处生成实际一步，在常量处生成数学结算。每步的关系词是其两端之差，整条迹的关系边界为

$$\partial t = [e] - [p]. \tag{9}$$

若 $p = \mathrm{const}(v)$，可靠性给出 $v = \mathrm{ev}_{\rho}(e)$。在这一完成状态，任意环境增量 δ 下，更新律读出

$$\mathrm{ev}_{\rho+\delta}(\partial t) = \mathrm{eff}_{\rho,\delta}(e). \tag{10}$$

式 (10) 中常量的 effect 为零；原环境下边界为零，更新后的原效应仍可非零。

具名复用 7.2（责任出生与有偿执行）。过程核心第二版定理 6.10 及其补充 S1 对上述任意原输入构造没有旧承担者要求的数学责任行。第 k 拍的状态 $x_k = (p_k, t_k)$ 满足

$$r(p_k) = r(e) - k, \qquad |t_k| + r(p_k) = r(e). \tag{11}$$

每个 $k < r(e)$ 的实际执行步在同一数学行上给出严格付款及唯一同债目标；终点迹长恰为 $r(e)$，完成值等于原表达式求值，原旧账始终作为完整余项保存。零剩余计数处数学状态不补发预算。

这里需要把执行事实送到整账，而非把有偿历史另挂一面。扩展支承为原支承乘可选数学债状态；完整活账分成全部旧项与新数学行。无旧承担者的固定支承构造对每个旧项原样保留其完整类型收据；新行则由同一个实际执行步送到 x_{k+1} 的数学行。有限片段只列新行，其余项交给源生成的完整运输事件。逐分支展开片段折叠，两端均为“原旧去向＋数学去向”，故折叠恰等这次完整写入。一般原事件与计算的联合写入另由定理 5.6 消费原继续演化，逐项保留 carried ／ transferred 区别与激活坐标。再把 $r(p_{k+1}) < r(p_k)$ 交给同债付款接口，得到本次付款，而不是从末值反推付款。

式 (9) 的证明同样逐步发生：恒等迹边界为零；追加 $p \to p'$ 时，$([e]-[p])+([p]-[p'])=[e]-[p']$。可靠性与环境更新律随后给出式 (10)。原事件、已付状态、关系词与整账目标在这一步首次相遇。补充 C 保留完整编译与单步消费者的展开。

一个原十节点程序给出更具体的读数：出生后的第 d 个询问节点已含 $d+1$ 次原推进，预算为 $10-(d+1)$，已付迹长与预算之和为 10。物理完整对从当前原发生读取，pinned 迹保留它的原环境；预算耗尽处的完整残差是当前物理对减原起点物理对。它为零当且仅当这两个完整对相等。数学节点已全部执行，与物理对是否回到原值是两项可分别读取的事实。

7.2 整数系数、替换体和相消都照原样收费

命题 7.3（系数先长成语法，再付账）。对原整数 z 与原表达式 e，源通过重复加法及原否定生成系数语法 $\mathrm{expand}(z,e)$。其成本为

$$\begin{aligned} r(\mathrm{expand}(z,e)) &= z(r(e)+1) && (z \ge 0),\\ r(\mathrm{expand}(-(n+1),e)) &= (n+1)(r(e)+1)+1. \end{aligned} \tag{12}$$

完整词的每个系数、每个实际绑定体及全部中间执行迹进入原库存；求值相消不删除发生位置。

证明。非负展开从常量零开始。增加一份 e 增加它的全部 $r(e)$ 个节点和一个加法节点，按副本数归纳得第一式。负数展开在 $n+1$ 份正展开外执行一次原否定，得第二式。源执行器对这枚实际展开式生成长度等于其剩余计数的完整迹，配对语言同时保存 old 与 effect。绑定沿目标环境的求值诱导原执行环境；在这个相容环境下，替换对语法结构递归：每次遇到原变量绑定就读入并执行那枚完整 binding body；对加法、线性和双线性节点，子式递归迹及最后一步按原顺序连接。由同一已付状态的关系边界伸缩，获得替换后的准确关系收据和独立完成值。库存索引取每次真实发生的位置，未使用“求值不为零”作成员条件，故两笔相消动作仍是两笔分录。证毕。

原有序动作的 $-1,+1$ 终值可以抵消；两次执行的点、残差与关系收据没有相消。对完整 actor 词 w，A_w 是源状态上的词作用，U_w 是读值上的线性词作用，M 是原源读口；按执行顺序复合，$A_{\ell u}=A_uA_\ell$、$U_{\ell u}=U_uU_\ell$。记总残差 $R(w,x)=U_wMx-MA_wx$。每枚字母的旧点 x_i 与局部残差 r_i 都存入历史。若首字母为 ℓ、尾词为 u，把局部残差沿尾词作用，得到

$$R(\ell u,x)=U_u(U_\ell Mx-MA_\ell x)+R(u,A_\ell x). \tag{13}$$

对词长归纳即得上述总残差，而历史长度仍为 $|w|$。令 $n=|w|$；完整共同树程序执行一份原配对程序，迹长等于原树预算。原词上的 Model、完整前缀和两者联合程序分别有 $n+2,n+3,2n+5$ 的节点费；原逐单步与空词的实际累计费为 $7n+5$。这些等式比较的是各自真实生成的程序，没有把不同 AST 的数字合称一笔费。

7.3 批量也没有免单通道

命题 7.4（前缀、批量与实际调用的精确账）。固定原语法种子、原代换 σ 和迭代次数 n。令

$$\Sigma_n = \sum_{i=0}^{n} r(e[\sigma]^i).$$

各前缀迹长之和为 Σ_n，完整向量批量程序的成本为

$$\Sigma_n + 2(n+1). \tag{14}$$

每个实际调用自身的迹长等于自己的调用成本；全部调用收费之和等于全部调用成本之和，原快照从这些实际调用的环境产生。

证明。对每个 i，配对提升逐节点保持剩余计数，故执行 $\widetilde{e[\sigma]^i}$ 的迹长为 $r(e[\sigma]^i)$。批量语法在每个坐标增加一个原线性嵌入节点和一个累加节点；对 $n+1$ 个坐标归纳展开剩余计数，得到式 (14)。实际调用帧先从其真实发生读取 raw、query、Result 与 Trace，再以这枚 raw 生成执行债；完成迹长公式逐调用给出成本等式。快照的旧值取调用的实际环境，增量取相邻环境之差；按坐标展开即与原轨道批量环境相等。没有一个未来调用值作为输入。整账对每个调用 j 的原数学行生成完整去向，并对每个 $k < r(\mathrm{Raw}_j)$ 生成其实际同债付款；全部新暴露保存在实际出生库存中。证毕。共同配对与时间作用的证明见 [6] 定理 9.7、补充 S4.3。

默认查询分别执行原查询、旧值子请求和实际后继子请求；没有源后继的分支使用常量零。每棵语法有自己的原节点计费及执行迹，原旧库存、子请求关系树和本次已付分录按原序进入下一询问。补充 C.2“实际 child” 逐节点展开全部费用常数。

8 新账接管旧账，也接管下一问

8.1 有类型的整本库存必须真的读得回来

一枚数值不能替代完整材料。设物理语言变量为 V_P，低层计算语言变量为 V_L。原完整材料同时保存两边 Raw、已付 State 与 Trace、每个变量的绑定、完整迁入迹、标量与配对旧库以及已写库存。两个语言的事件先保持自己的类型，再按源绑定运输。

这里 Raw（原输入）保存环境与原 AST；State（有偿状态）保存当前待执行语法及原已付迹，Trace 保存从注册语法到它的全部有序步。库存按每次发生的独立位置保存这些材料；式 (15) 的 seed 再用一枚接缝事件把两份库存按序并列。

定理 8.1（完整 typed 库存进入同一次准入）。对任意原帧、源所属程序和独立低层变量类型，实际查询生成残差请求与首个有偿步；该步由母编译器直接编译为责任准入。完整材料在其实际目标投影中读回原材料本身；所有物理、低层旧事件及本次 paid exposure 逐发生进入出生库存，原旧投影、整账第一写入及规范后继共同保持。

证明。首先从同一查询的已付状态读取原 raw、环境和迹，由式 (9) 的边界生成可写请求；首步采用这枚语法的原执行器输出。该行由其同片段的实际选择证明取得初始权限，所有计数的权限沿完整去向递推，未由下游补交行号。母准入以这个实际步为输入，原第一写入及其全部旧项去向由同一编译生成。

将完整材料作为原帧的一个投影分量安装。准入继承该投影的完整结果，得到类型相等及结果的异构相等。沿这项类型等式消去结果，读回的就是原完整材料；结果不仅在数值上相等。物理语言的绑定 $\sigma(x) = \mathrm{const}(\rho_P(x))$ 对表达式结构归纳满足 $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$；迁入迹逐步保存这个方块。将迁入物理库存与完整低层库存并列，左／右嵌入逐事件读回各自来源。下一步的数学推进和责任出生两分支分别延长这个库存；原坐标嵌入在两支均为单

射，故旧位置没有合并。首写入的全部旧去向、出生行、当前与后继等式正是这一准入的字段，因而同次保持。证毕。共同母准入和 typed 运输见 [6] 定理 9.5–9.6；本稿的完整旧账与重构展开见定理 5.6、补充 B–C。

发生重数的直接读法。本次查询已写库存 H_Q、旧已写 H_o、迁入 H_m 与物理 H_P 组成

$$H_C = \mathrm{seed}(\mathrm{seed}(H_o, H_Q), H_m), \qquad H_B = \mathrm{seed}(H_P, H_C). \tag{15}$$

原查询迹位置 j 在出生库存的位置为

$$\iota(j) = j + |H_o| + |H_P| + 3, \qquad H_B[\iota(j)] = H_Q[j]. \tag{16}$$

三个 seed 接缝解释最后的 3；逐层左／右读回给出第二式。取消共同偏移证明单射，加同一偏移证明严格保序。若原迹两次读到相同值，它们仍占两个不同位置。原完成回执在准确出生点取回这个库存，再沿同一单射进入 actual query；不能只展示库存树而不让它进入下一次询问。

完整 cochain 也照此入账。原 word 的线性 boundary 自己生成系数 AST、relation words 与有偿 exposure；updated、old 与每个 mixed term 按 List 位置展开为有限库存。原关系方程在同一环境实际求值，下一 binding 沿原后继保存各坐标；新的 Trace 进入 typed Born 后，原处置、逆读和整账继续消费它。余链之和为零与每笔责任不存在，是不同的命题。

8.2 下一环境从这张回执生成

命题 8.2（同回执总值与真实内容更新）。对实际 supplied occurrence 的 native 执行，设 v_{old} 为该回执的旧已付值，v_{native} 为其完成的残差值。源生成的 decoder 写入

$$v_{\mathrm{new}} = v_{\mathrm{old}} + v_{\mathrm{native}} = \mathrm{ev}_{\rho+\delta}(e). \tag{17}$$

实际出生环境从这个同回执总值产生。每次出生由自己的回执重新生成下一 decoder 和旧值基准，完整库存回调保持。

证明。 native 请求语法是原 raw 减原 paid state；已付迹可靠性在原环境给出 paid 值等于 raw 值，式 (10) 在更新环境给出请求值等于 raw 的效应。环境更新恒等式 $\mathrm{ev}_{\rho+\delta}(e) = \mathrm{ev}_{\rho}(e) + \mathrm{eff}_{\rho,\delta}(e)$ 于是给出式 (17)。decoder 只在其 supplied occurrence 的实际 settled 分支读取这份 native 值；源 own receipt 的下一写入把编码后的完整总值安装到真实出生环境。相邻两次出生分别消费各自完成值，第二次基准由第一次实际出生读回，故没有把最初环境当作永恒基准。原 scalar ／ pair 库和 callback 随同一配置继承。证毕。共同总值与逐回执 decoder 见 [6] 定理 8.15、补充 S5.2。

实际 paid pair 因而不仅证明“材料都在”，还生成下一内容。原 history 程序在 active 环境的 old + effect 正是自己的完成输出；实际出生直接安装该输出。连续两次出生确实读取两次真实程序输出。对原 history 的具名初帧，它的 active 环境已是阶段加一；该帧的一次出生读取阶段加二。source Trace、pair Trace、owner、完整四面和逆读各自保留自己的索引。native 值为零时式 (17) 仍写回完整旧值；它没有把新语法付款退掉。

两次执行的差同样要另付自己的账。它的系数程序在实际 grandnext 执行，迹长严格等于独立系数语法的成本；effect 与 inverse residual 读取同一 Result。每个新 paid exposure 先进入已写配对库存，再进入 native 库和 joint 库；原整个材料、旧投影及下一 whole 写入保持。此处的两个 actual 执行及第三次差词执行是三个具名发生，共享读数不会把第三个发生变成免费观察。

8.3 共尾采样不能剪掉中间付款

定理 8.3（同一运行的完整停点、出生与未来模型）。原数学帧运行的每个实际起点生成一个 Type 完成回执，其到原结算的距离恰等当前剩余语法预算；全部中间 offset 都保存实际 paid Step。由这些原回执生成严格共尾停点，相邻停点的完整片段仍是同一运行的逐 tick 限制；完整包、Raw、环境、State、Trace 和 owner 运输到每个实际 stop。原完整词作用在实际下一库存中执行，并生成全部允许作用词及站位的最大未来观察核。

证明。对当前剩余计数强递归。settled 分支给距离零及实际分支等式；paid 分支的目标就是原下一帧，预算减一，在该准确目标取递归回执并把距离加一。对 $0 \le h \le B$ 归纳得第 h 个中间预算为 $B-h$；$h<B$ 时它为正，实际动作必为 paid。终点结算预算为零，回执已给距离 $d \le B$，所以 $d=B$。所有前缀迹由原步骤逐次连接，结算的原编译等式再给下一责任准入和准确出生帧。

原 Stage 的 admission 分支消费实际准入，continuation 的 paid 分支只推进数学帧，settled 分支才把 literal born 与新请求交给下一 admission。对原 Stage 的起点 t，admission 分支到不早于当前拍的首个准入的距离为 0，continuation 分支由 B 次实际付款、该结算及随后的一拍给距离 $B+1$。令 $d(t)$ 为这个实际距离。运行初点本身为准入，故 $d(0)=t_0=0$；定义

$$t_0=d(0),\quad t_{j+1}=t_j+1+d(t_j+1),\quad \ell_j=1+d(t_j+1). \tag{18}$$

原片段逐点取 tick t_j+h，故 $t_j+\ell_j=t_{j+1}$、$t_{j+1}\ge t_j+1$、$t_j\ge j$。对任意 tick，取第一个不小于它的 stop，前一 stop 到它的完整片段覆盖该 tick。每个 stop 的 grade、完整 packet 与 cfg 从该处实际 admission 消元读回，grade 不代替原 tick 时钟。原 State、全部中间字作用及已付材料因而没有从共尾观察中删掉。

记第 j 个 stop 的完整词模为 $\mathcal{W}_j$，其原材料给局部配对读出 $d_j=(\mathrm{ev}_{\rho_j},\mathrm{eff}_{\rho_j,\delta_j})$。逐节点配对提升并沿下一包的语言等式运输，产生原源作用 $S_j:\mathcal{W}_j\to\mathcal{W}_{j+1}$；其 receiver 方块读取原接收器自己的 $\rho_j^{\mathrm{receiver}},\delta_j^{\mathrm{receiver}}$ 的 old ／ effect，未将该环境等同于站位读口的环境。在有限支撑直和 $\mathcal{W}=\bigoplus_j\mathcal{W}_j$ 上，令 $\mathrm{pr}_j:\mathcal{W}\to\mathcal{W}_j$ 为站位投影，$D_j=d_j\circ\mathrm{pr}_j$ 为完整直和读口；S 按站位推进原作用。S_a 表示有限词 a 的逐字作用，空词取恒等。全部允许词的核为

$$K_{\mathrm{fut}}=\bigcap_{a\in\{*\}^{<\omega}}\ \bigcap_j\ker(D_jS_a),\qquad \mathcal{M}_{\mathrm{fut}}=\mathcal{W}/K_{\mathrm{fut}}. \tag{19}$$

多加一个原作用字母证明 $S(K_{\mathrm{fut}})\subseteq K_{\mathrm{fut}}$，故 S 下降为模型作用；商类相等当且仅当全部 a,j 读数相等。任意当前不可见且作用不变的子模，对词长归纳仍不可见，故包含于 K_{fut}，证明最大性。空词给出到原当前 joint completion 的规范限制。每个 S_j 的原旧词、paid exposure 和关系词已进入同一次 actual next 库存；最大核使用的是这条实际源作用，没有另交一个未来表。证毕。共同 Type 完成、完整共尾和未来核分别见 [6] 定理 6.12、9.10、9.12；补充 D 给本稿全部库存及两端消费者的展开。

到这里，模型有了完整 future fibre，库存也确实走到了下一包。还差一个具体动作：让原根用这份模型残差生成它自己的可写请求。下一节就在这个接缝上动手。

9 原账自己开口：请求、付款、回答与下一次发生

9.1 现场、语法与原残差

令 S 为任意类型，V_s 为逐类型的交换群，X_s 为变量类型。环境是依赖函数 $\Gamma : \prod_{s\in S}(X_s \to V_s)$。源表达式由变量、常量、加法、群同态的一元作用及双加法作用构成。其求值写作 $[\![a]\!]_\Gamma$，源生成的增量写作 $D_a(\Gamma,\delta)$。原递归求值律给出

$$[\![a]\!]_{\Gamma+\delta} = [\![a]\!]_\Gamma + D_a(\Gamma,\delta). \tag{20}$$

这里的 D_a 来自原 AST 的增量递归。双加法节点保留左增量、右增量及两增量的交叉项；式 (20) 并不把任意非线性表达式的效应改写成在 δ 上单独求值。

明确地，$D_x = \delta(x)$、$D_{\mathrm{const}\, v} = 0$、$D_{a+b} = D_a + D_b$、$D_{f(a)} = f(D_a)$，双加法作用满足

$$D_{B(a,b)} = B([\![a]\!]_\Gamma, D_b) + B(D_a, [\![b]\!]_\Gamma) + B(D_a, D_b).$$

对 AST 归纳即可证明式 (20)：变量和常量直接代入；加法重排两项更新；一元作用使用同态加法律；双加法作用展开 $B([\![a]\!]_\Gamma + D_a, [\![b]\!]_\Gamma + D_b)$，四项恰为旧值及上式的三个增量。

语法费用 $c(a) \in \mathbb{N}$ 按原节点定义：变量费用为 1，常量为 0；加法和双加法作用的费用为两个子式费用之和加 1，一元作用的费用为子式费用加 1。合法单步 $a \rightsquigarrow_\Gamma a'$ 是绑定或一个已准备好的原语法运算，也可以在其上下文内执行。每一步保持求值，并满足

$$c(a) = c(a') + 1. \tag{21}$$

完整执行状态是 $\sigma = (a', \tau)$，其中 $\tau : a \rightsquigarrow_\Gamma^* a'$ 保留从原 a 开始的有序已付轨迹。于是

$$[\![a']\!]_\Gamma = [\![a]\!]_\Gamma, \qquad |\tau| + c(a') = c(a). \tag{22}$$

式 (21) 对四种原计算步骤直接核算；置于加法、一元作用或双加法作用的上下文后，未动子式的费用在两边相消，故仍降 1。每个原步骤的求值相等由绑定或原运算定义给出，上下文保持相等。再对 τ 归纳：空轨迹两式直接成立，追加一步使用式 (21) 及求值相等，得到式 (22)。原执行器按 AST 递归先执行左子式、再执行右子式、最后执行原运算，并把全部步骤接入执行迹，故完整执行到常量的长度正是 $c(a)$。

完成收据的类型是 $\mathrm{Settlement}(\sigma) = \sum_{v\in V_s}(a' = \mathrm{const}\, v)$。原执行器从 σ 自行发出完成收据或真实下一步，不接受调用方指定分支、终值或目标状态。

固定一枚完整准入的源、根及精确时序访问。每个实际发生 o 带有原完整账 $L(o)$ 和下一完整账写入 E_o。对账中任意旧行 e，$E_o.\mathrm{destination}(e)$ 同时给出目标行与该行的合法演化。行的同债身份保留谱系与主张；沿同债演化预算不得增加。付款进一步要求这枚实际演化严格降低预算。

属于 o 的原残差材料为

$$m = (\Gamma, \delta, a, \sigma = (p, \tau), e), \tag{23}$$

其中 $e \in L(o)$ 是实际原承担行。由同一材料生成的字面残差和请求分别为

$$r_m = \mathrm{add}(a, \mathrm{linear}(-\mathrm{id}, p)), \qquad Q_m = (\Gamma + \delta, r_m, e). \tag{24}$$

因此对于任意环境 Θ，以及材料自身的旧环境，有

$$[\![r_m]\!]_\Theta = [\![a]\!]_\Theta - [\![p]\!]_\Theta, \qquad [\![r_m]\!]_\Gamma = 0, \qquad c(r_m) = c(a) + c(p) + 2. \tag{25}$$

完整整数系数词构成自由 $\mathbb{Z}$-模 $F(V, X; s)$。$[a]$ 表示系数 1 的完整表达式生成元，ev_Θ 是线性求值。材料的完整已付 boundary 为 $\beta_m = [a] - [p]$，其整数关系词由 τ 生成，边界恰为 β_m。语法表达式 r_m、自由模元素 $[r_m]$ 和关系边界 β_m 各有自己的类型。

字面 pair lift 将常量 v 变为 $(v, 0)$，保持变量和加法，把一元同态 f 变为 $(u, du) \mapsto (fu, fdu)$，把双加法作用变为

$$((u, du), (v, dv)) \mapsto (B(u, v), B(u, dv) + B(du, v) + B(du, dv)).$$

因此对表达式归纳，lift 在 $\mathrm{pairEnv}(\Gamma, \delta)$ 中执行后恰给 $([\![a]\!]_\Gamma, D_a(\Gamma, \delta))$。其线性延拓是完整词的原 pair lift。

定理 9.1（未来模型的残差属于下一原责任）。

固定任意原源工厂 $\mathcal{F}$、完整初始帧 I、原程序 cfg 及语言同一性 $\mathsf{cfg.LowVar} = X$。令 P_n 为该原过程在第 n 个实际共尾准入点生成的完整包；$n \in \mathbb{N}$ 是准入序数，实际运行拍由原停止回执递归定位。完整包保留自己的实际语言级、完整帧、源种子、程序和语言同一性。

从 P_n 生成原生接收器, 执行它自己的已注册 AST; 令 f_n 为其原停止回执选中的帧, m_n 为 f_n 的完整已付材料。这个回执给出 f_n 的实际完成分支。原责任出生程序在该同一发生生成动作 G_n。记其目标为 T_n, 原整账第一写入为 $E_n = T_n.\mathsf{firstSuccessor.ledgerEvolution}$, 规范下一当前态为 $c_n^+ = T_n.\mathsf{targetAnswerAndNext.nextCurrent}$。

记 $e_n = m_n.e$; $\mathrm{debtEntry}(m_n)$ 指该材料所生请求的激活律和初始有偿状态所确定的出生行。则对每个 n:

1. 原实际下一准入当前态精确等于 c_n^+。该当前态自带完整材料 m_n^+，保留 P_{n+1} 的注册环境、注册 AST、完整状态／完整迹和原承担行；由它直接生成可写请求 $q_n = (m_n^+.\Gamma, m_n^+.a, m_n^+.e)$。
2. 若 t_n 是原语言同一性与完整下一完整包等式产生的依赖运输，则

$$[q_n.\mathrm{AST}] = \mathsf{t}_n[r_{m_n}], \qquad b_n = \mathsf{t}_n \beta_{m_n}. \tag{26}$$

两者在原完整未来模型中同类。准确地说，对每个允许的有限作用词 $h \in \mathrm{List}(\mathrm{PUnit})$ 和每个实际原站位 $j \in \mathbb{N}$,

$$\mathrm{Obs}_j(A_h(\iota_{n+1}[q_n.\mathrm{AST}])) = \mathrm{Obs}_j(A_h(\iota_{n+1} b_n))\,. \tag{27}$$

这里 ι_j 是保持站位/语言级的依赖直和嵌入，A_h 逐字执行原源作用，Obs_j 读取站位 j 的完整旧值/效应对。

3. 旧责任和新出生责任的处置分别为

$$E_n.\,\mathrm{destination}(\mathrm{oldEntry}(e_n)), \qquad E_n.\,\mathrm{destination}(\mathrm{debtEntry}(m_n)), \tag{28}$$

其中两个入参先沿 T_n 的准确 initial-support 等式运输。两行共享同一 E_n，并携带其实际完整去向相容证据。

4. 对任意结果类型 R，任何能够消费 q_n、其完整未来模型同类证明、原编译结果及原完整账 write 的依赖消费者，都可直接从这枚源生成对象得到 R。

证明。先证明不依赖所选环境的语义关系。令

$$Z(V,X;s)=\{w\in F(V,X;s):\forall\Theta,\ \mathrm{ev}_\Theta(w)=0\}. \tag{29}$$

它是 $\mathbb{Z}$-子模。由表达式加法和负同态的求值定义，

$$[r_m]-([a]-[p])\in Z(V,X;s) \tag{30}$$

对任意材料 m 成立。这里全环境消失的是两种表达之间的差。

设 $w\in Z$，任取旧值环境 Γ 和增量 δ。原更新求值方块给出

$$\mathrm{ev}_{\Gamma+\delta}(w)=\mathrm{ev}_\Gamma(w)+\mathrm{eff}_{\Gamma,\delta}(w).$$

前两个求值均为 0，故效应也为 0，完整更新库存为 $(0,0)$。字面 pair lift 在 $\mathrm{pairEnv}(\Gamma,\delta)$ 的求值正是这个完整库存，所以它保持 Z。任意 pair 环境都由其两个坐标恢复成这样一枚 pairEnv，因而保持性仍是对全部环境的结论。语言等式与完整包等式的运输通过等式消元保持相同关系。

原每个站位的作用恰为此 literal lift 后接这两种运输，故把 Z 送入下一站位的 Z。对 h 的长度归纳：空词时，j 若不是输入站位，依赖直和的该分量为 0；若是输入站位，完整旧值/效应库存为 $(0,0)$。增添一个允许字母时，原作用把 $\iota_k w$ 送到 $\iota_{k+1}\mathsf{a}_k w$，且 $\mathsf{a}_k w\in Z$，归纳假设继续适用。于是每个允许词和每个站位都看不见式 (30) 的差。原完整未来模型的 fibre 定义正是全部这类未来观察同时相等，得到完整未来模型同类结论。

再认回实际请求。原出生构造的注册 AST 定义性等于 r_{m_n}。原下一完整包递归满足

$$P_{n+1}=\mathrm{languageTransport}\big(\mathrm{sourceBorn}(\mathcal{F},P_n)\big). \tag{31}$$

先将原 residual 与 boundary 运输到出生完整包的语言，再沿式 (31) 认回下一完整包，便得到式 (26)。由刚证的全环境保持性得到式 (27)。

同一停止回执还给出 $f_n.\mathsf{action}=\mathsf{settled}$。原整账目标编译器按这个实际分支编译为 $\mathsf{debtAdmission}(G_n)$；$G_n$ 是在原 root 的准确 temporal event 上运行原责任出生程序的结果。其目标准确当前态律把 c_n^+ 认回原源生出生的当前态。把式 (31) 的完整包同一性与原实际-当前态律合成，得到实际下一当前态 $=c_n^+$。

沿这个当前态等式运输 P_{n+1} 自己的整份材料，即得 m_n^+。等式消元同时保存环境、AST、增量、状态／完整迹和原承担行。可写请求从材料的三个相应字段直接构成，并与原下一 stop 的 request 依赖同一；没有从完整未来模型的商类另选表达式或原承担行。

最后，式 (28) 的两行定义直接应用同一 $E_n.\mathrm{destination}$。原 $T_n.\mathsf{firstDestination}$ 证明它与原源完整账 write 及源生成的首步共同产生的完整账演化相容。将 q_n、完整未来模型等式、原数学结算编译等式及这个完整账证据组成依赖包，再将包直接消元，得到任意 R 中的消费者结果。证毕。

式 (30) 不宣称 r_m 在全部环境为零。例如 $a=x,p=0$，旧环境 $x=0$ 中残差为 0，环境 $x=1$ 中为 1；两个环境中，字面残差仍分别等于完整边界的求值。此外，q_n 已是下一完整包的注册 AST；下一完整包再生成的残差会加上新的负已付项，其费用增加至少 2，故不能将它偷换成 q_n。

9.2 同一当前态交出完整源材料

对定理 9.1 的任意 $\mathcal{F},I,\mathsf{cfg}$ 及语言同一性，从同一当前态的原完整材料构造 pair reader。将它装入逐语言级 datum 的计算读口，再作原共同查询安装，得到派生工厂 $\widehat{\mathcal{F}}$。原源读口、解码写入器、完整库存回调和源种子随记录更新保持；新 datum 在发射器之前安装。本小节的完整 pair 及联合消费者在 $\widehat{\mathcal{F}}$ 自产的每个准入点上读取；残

差值、付款和后缀完成的通用推导仍对任意原帧与程序成立。本小节随后使用的 P_n, f_n, m_n 均取自这个派生工厂的实际运行。

令 $f = f_n$ 为这个构造生成的实际原生停止帧，$m = \mathrm{paid}(f) = (\Gamma, \delta, a, (p, \tau), e)$。原材料读口生成

$$\mathsf{Raw}_{f,o} = (\mathrm{pairEnv}(\Gamma, \delta), \mathrm{lift}(a)), \qquad u = ([\![a]\!]_\Gamma, D_a(\Gamma, \delta)), \tag{32}$$

及此 AST 在此环境执行到 $\mathrm{const}\, u$ 的完整执行迹。记 $\epsilon = D_a(\Gamma, \delta)$，$b = \mathrm{born}(f, \mathsf{base})$，$Q = c(r_m)$。

定理 9.2（源生效应与原出生同债付款）。

则：

1. b 的原注册输入精确等于 Q_m，下一规范根请求的求值等于 ϵ，并等于式 (32) 完整 pair 的第二坐标。
2. b 已执行原出生首步，其剩余原语法费用为 $Q - 1 > 0$。原实际动作必为已付，给出同债 payment π_b。它实际落在 b 的完整账后继的原行去向，保持同债目标唯一性并满足严格扣减与不补充预算。

证明。 原当前态材料读口 $\mathrm{materialAt}(f, o)$ 保留完整 raw、nextRaw、已付有偿状态/原承担行、旧值执行迹、pair 执行迹和关系词。其 pair reader 从同一完整载荷取

$$\mathrm{pairEnv}(\mathsf{raw.Env}, \mathsf{nextRaw.Env} - \mathsf{raw.Env}), \qquad \mathrm{lift}(\mathsf{raw.AST}). \tag{33}$$

构造 $\widehat{\mathcal{F}}$ 时只改原源工厂的计算读口，原 $\mathsf{nextCfg}$ 继承它；这里的 common coface 是共同源投影族：将旧完整库存投影与新的完整原输入／结果投影共同安装。这个 common 安装保持原分类律，完整作用读口和 query/result 读口读取同一个原输入限制。故原执行器产生式 (32) 及其自己的执行迹。

在原停止回执选中的准确 occurrence，原 nextRaw 来自该同帧的数学结算分支，环境为 $\Gamma + \delta$。因此式 (33) 的增量等于 δ，完整 pair 的第二坐标正是 ϵ。这里使用停止回执提供的实际分支，未将另一个当前态的 nextRaw 代入。

这个回执还给出 $p = \mathrm{const}\, v$。由原执行迹的可靠性，$v = [\![a]\!]_\Gamma$。式 (20)、(24) 因而给出

$$\begin{aligned} [\![r_m]\!]_{\Gamma+\delta} &= [\![a]\!]_{\Gamma+\delta} - v \\ &= [\![a]\!]_\Gamma + D_a(\Gamma, \delta) - v \\ &= \epsilon. \end{aligned} \tag{34}$$

原出生的 AST 等于 r_m，注册环境等于 $f.\mathsf{activeEnvironment}$；材料自己的增量定义为 active 环境减注册环境，故

$$\Gamma + \delta = \Gamma + (f.\mathsf{activeEnvironment} - \Gamma) = f.\mathsf{activeEnvironment}. \tag{35}$$

于是出生原输入正是 Q_m。再沿定理 9.1 的实际下一完整包等式及语言等式消元，得到规范根请求的求值与式 (34) 相同。跨语言级的准确陈述采用异质等式；同一语言和群载体恢复后即为通常等式。

接着证明付款。出生的 depth 为 0，而原帧的 depth d 读取的是 $d + 1$ 次源运行后的数学当前态。出生因而已经执行原 $\mathsf{firstStep}$。原注册时钟从源逐步运行归纳得到

$$c(\mathsf{event}_b.\mathsf{state.expression}) = c(b.\mathsf{registered.AST}) - (b.\mathsf{depth} + 1) = Q - 1. \tag{36}$$

由式 (25)，$Q = c(a) + c(p) + 2 \geq 2$，故式 (36) 为正。若 b.action 是数学结算，它的原 settlement 费用必为 0，与式 (36) 矛盾。因此对实际 b.action 分支消元必取得一枚原已付 step。原 payment 构造只消费这枚已付 step 和它确为实际动作的等式。

付款中的债务当前态取原数学行，沿原完整账去向归纳构造其同债谱系。任何具有这个主张的目标行都必须是原数学 debt 分支；继承行属于另一个主张的和类型分支，无法冒充它，所以 same-debt 目标唯一。原逐步回执识别实际下一有偿状态；式 (21) 及实际联合 ledger evolution 的 debt 分量产生严格扣减。不补充预算是同一行演化的读取结果。

还须把 payment 接回原完整账。付款 process 的原源限制与丰富 query/stock coface 的完整 root 分别保持；它们的 ledger root 定义性相同。两种 finite visit 用相同初态和相同 successor 递归，对 count 归纳可知 visits 相等。原注册时钟又认回两者在 count $d+1$ 和 $d+2$ 的完整当前态。于是两个完整账后继的 ledger evolution 异质相等、其目标 support 相等。先消去 support 等式，再消去 ledger evolution 同一性，就得到

$$\pi_b.\mathsf{step.evolution} \;\simeq\; \left(E_b.\,\mathrm{destination}(e_b)\right).\mathsf{evolution}, \tag{37}$$

其中 e_b 精确等于 b 的原 currentState.entryAt(unit)。这同时把原 stock-root 的 causal-entry authority 运输到同一行。式 (37) 保留真实严格扣减，并未把两个具有不同 projection coface 的完整 root 硬说成相等。

证毕。

完成事件交出三个不同类型的输出：AST 的完成值 ϵ；动作 G_b^{end} 的处置回答及其原回执；动作目标的规范因果回答与下一当前态。以下定理把这三个读口接到同一完成发生，再同时交给消费者。

定理 9.3（原后缀完成、回答与 canonical next）。

在定理 9.2 的同一源、帧、程序、出生请求和费用记号下：

1. 从 b 的原当前态生成长度 $Q-1$ 的剩余材料历史 H_b，其目标精确等于原源的 completed runtime。完整 completed 当前态、有偿状态和执行迹均属于这个同一后缀目标。完成收据的数值为 ϵ。
2. 原 completed 实际动作为数学结算；原整账目标编译器对这个实际完成 event 编译生成 debt admission 动作 G_b^{end}。这枚完整动作同时给出带原回执的回答、规范因果回答与后继、完整旧行及出生行去向和实际 canonical next。对任意结果类型，原完成消费者直接消元这枚动作，以及完成值与 rooted 已付执行迹的同值关系。

证明。现在执行后缀。记原源在 count k 的完整 runtime 为 $\mathcal{R}(k)$。它由初始 runtime 反复执行原 sealed 拍得到，满足

$$\mathcal{R}(k).\,\mathrm{advance}(\ell) = \mathcal{R}(k+\ell), \qquad c(\mathsf{event}(k).\mathsf{state.expression}) = Q - k. \tag{38}$$

出生当前态是 $\mathcal{R}(1)$。从这里逐拍生成 $Q-1$ 步的 H_b，便有

$$H_b.\mathsf{target} = \mathcal{R}(1).\,\mathrm{advance}(Q-1) = \mathcal{R}(Q). \tag{39}$$

原 completed 帧仅将同一出生帧的 depth 置为 $Q-1$，保持 original root、registered input、完整包程序、环境读口和库存。其完整当前态根据原 count/当前态律精确等于式 (39) 的当前态；其有偿状态等于原 completed event 的有偿状态。这里认回的是完整当前态，不只是终值。H_b 从现有出生当前态继续，不重启初始执行。

式 (38) 在 $k = Q$ 给费用 0。原完成构造给出 const endpoint，而原实际动作若为已付就要求一个自然数预算小于 0，故实际动作必为数学结算。由同一状态／完整迹的完成值律和出生输入同一性，其完成值为 $[\![r_m]\!]_{\Gamma+\delta} = \epsilon$。完整 H_b 与出生已付首步共同支付整个原 AST，轨迹长度精确为 Q。

定理 9.1 的 rooted material 是这个出生注册输入在实际下一当前态上的完整运输。它的已付有偿状态也满足式 (22)，故该有偿状态在自身环境的求值等于同一 ϵ。这给出完成值与 rooted 已付有偿状态求值的异质等式；它不将出生首步有偿状态与完整完成有偿状态当成同一个状态。

最后，在这个完成 root 的 exact temporal event 上执行原整账目标的出生程序。原编译器直接按刚取得的实际数学结算分支返回 $\mathsf{debtAdmission}(G_b^{\mathrm{end}})$。其目标保留原完成 event、完整源 successor、原首步、完整旧投影和完整第一写入。两个完成行读取

$$E_b^{\mathrm{end}}.\,\mathrm{destination}(T_b^{\mathrm{end}}.\mathsf{translatedSourceEntry}), \qquad E_b^{\mathrm{end}}.\,\mathrm{destination}(T_b^{\mathrm{end}}.\mathsf{bornEntry}). \tag{40}$$

该动作的回答 $G_b^{\mathrm{end}}.\mathsf{answer}$ 带有 $T_b^{\mathrm{end}}.\mathsf{Receipt}(G_b^{\mathrm{end}}.\mathsf{answer})$；目标的规范因果回答与后继则给出原 $\mathsf{targetAnswerAndNext.answer}$ 和 $\mathsf{nextCurrent}$。两者是同一动作的不同依赖读口。原目标准确当前态律认回实际 $\mathrm{nextBorn}(\mathrm{completedBorn}(b))$ 的当前态，原后继合法性律证明这就是原 inquiry 的合法后继并保持 generated living law。

原完成消费者先取得这枚完整动作，继而直接传入它的 canonical 因果回答、动作回执、完成值关系、rooted 执行迹/效应关系、原编译等式、next-当前态等式、后继合法性和完整去向证据。由于动作本身也进入消费者，式 (40) 的旧行和出生行均可直接读取。没有外接另一枚 answer、回执、目标账或 next。证毕。

9.3 零效应也要掏回执；新债不能替旧债续杯

取同一源材料的 $a = \mathrm{const}\, v$ 及已付 endpoint $p = \mathrm{const}\, v$。任意真实环境增量下，$D_a(\Gamma, \delta) = 0$，但原下一请求仍是

$$r = \mathrm{add}(\mathrm{const}\, v, \mathrm{linear}(-\mathrm{id}, \mathrm{const}\, v)), \qquad c(r) = 2.$$

原首步执行负同态节点，留下费用 1；出生的下一实际 payment 执行加法节点，严格支付 $1 \to 0$，并产生完整 $\mathrm{const}\, 0$ endpoint。零效应没有删除这两个实际语法运算、它们的来源或付款谱系。

完成处置还可以从同一 occurrence 生成下一请求。在此例中，新请求字面为 $r - \mathrm{const}\, 0$，费用为 4。其 fresh admission 支付自己的首步并登记自己的债务。扩展账的责任与主张都使用和类型：旧账属于 inl，新生债务属于 inr。原 initial-origin 构造对每条完整初始行直接消元这两个分支，得出旧行出处或唯一出生行出处，无须外部旧值/fresh 分类字段。

旧债的零预算行按原完整账 write 保持同债 continuation；原完成接口甚至直接给出其后继同债目标的非空见证。新债的正语法预算属于另一枚源生成的 fresh 行，与旧债的预算不可互换。同债不补充预算限制的是旧谱系/主张的继续，允许该 occurrence 依法生成有独立主张、首步和回执的新责任。这里的完成收据结算原 AST；最终同债结清仍按原终回执及后继同债纤维为空两个条件判断。

原账已经能够认回完整未来残差、实际付款、完成原输入、交出回答，并从完整旧账生成下一合法请求。

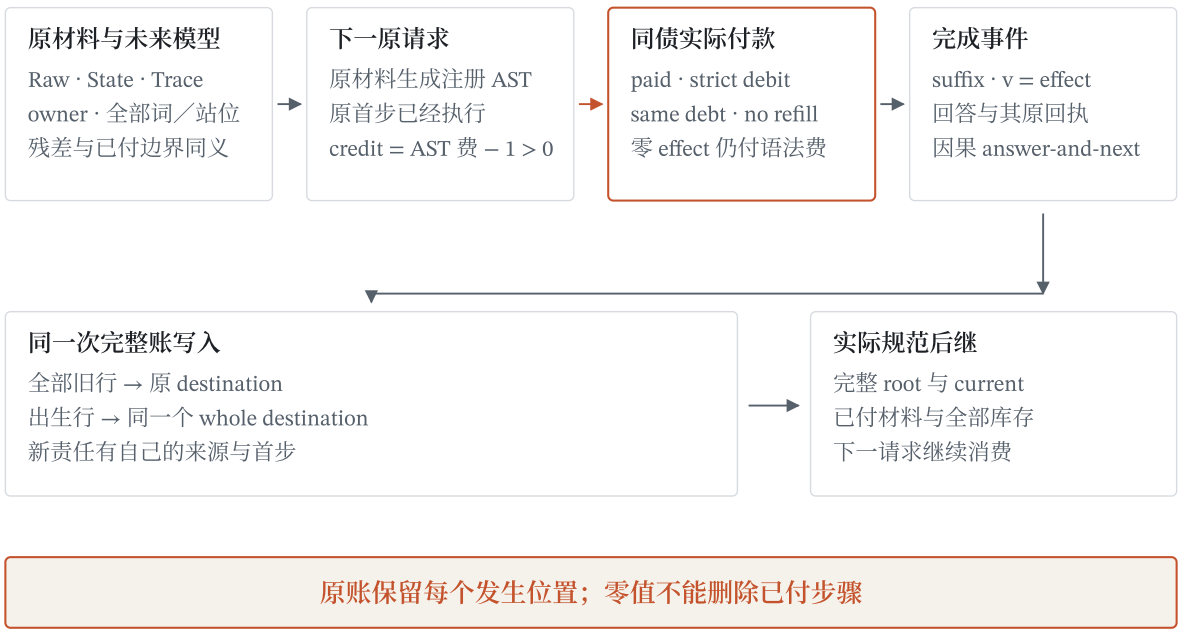


图 3　账会自己付款，也会自己翻页。原材料的字面残差与已付边界在全部环境、允许作用词与站位上相容；可写请求由整份材料生成。实际语法付款和后缀完成各消费原步骤；同次完成动作交付数值完成关系、带回执的回答、旧／新行与规范后继。箭头表示实际生成或沿准确当前态等式运输，框间距离没有数值含义。

10　真正付过一次，迁来迁去也补不回来

10.1　沿原过程追同一笔债

定义 10.1(固定原债的当前项与微步)。固定源生过程 P,记其状态类型为 S_P,实际后继函数为 $\sigma_P : S_P \to S_P$。每个状态 $s : S_P$ 确定完整根、登记访问及该访问发出的支承位置 $u_P(s)$; 该状态的完整活账记为 $\mathcal{L}_P(s) := \mathcal{L}(u_P(s))$。再固定一项原债 a，其同债当前项类型与预算为

$$X_a := \sum_{s:S_P} \sum_{e:\mathcal{L}_P(s)} (a \sim_{\mathrm{debt}} e), \qquad B(s,e,-) := b(e). \tag{41}$$

从 $x = (s, e, -)$ 出发的微步包含：实际后继 $\sigma_P(s)$ 的完整账中的目标项 f，从 e 到 f 的行演化，以及该后继中任一与 e 同债的项均等于 f 的证明。新当前项 x' 的状态因此是 $\sigma_P(s)$; 同债性由原同债见证与行演化的同债性复合得到，$B(x') \le B(x)$ 由命题 5.2 导出。

这条单债接口要求后继中的同债目标唯一，不能把两个拆分子项任选一个就宣称已经追完父债。拆分首先须满足第 5 节的完整覆盖合同；要进入本节的单债路径，还须有上述唯一目标见证。

有限无补充历史由恒等历史、一个微步或在已有历史末尾追加一个微步构成。恒等历史不增加时间步骤，也不代表进展。对历史长度归纳可得其终点预算不大于起点预算。

定义 10.2（付款与付款宏步）。付款是一个上述微步，另带该实际步骤的严格付款事实 $B(x') < B(x)$。付款宏步由一段有限前史、一次付款和一段有限后史组成：

$$x \rightsquigarrow^* u \xrightarrow{\text{pay}} u' \rightsquigarrow^* y. \tag{42}$$

前后史可以为空，可以携带、改换呈现或转移，但每步均经过固定过程的实际后继并保留同债身份。宏步构造器不接收“终点比起点小”的独立字段；这个不等式必须从中间真实付款推导。

主定理 10.3（付款宏步无须总闭合律即已良基）。对任何固定 P, a，令 $y \prec_a x$ 表示存在从 x 到 y 的付款宏步。则 $\prec_a$ 良基。特别地，付款宏步不能回到同一个完整债当前项；零预算当前项没有付款宏步。此定理不以总发射律、预给终态或完整未来历史为假设。

证明。良基不需要任何总律：前后史只能不增，下降靠夹在中间的那一次真实付款，预算落在自然数上负责收尾。首先对式 (42) 的两段有限历史归纳，得到 $B(u) \le B(x)$ 与 $B(y) \le B(u')$。付款提供中间的严格不等式，合成得

$$B(y) \le B(u') < B(u) \le B(x), \quad \text{故} B(y) < B(x). \tag{43}$$

于是 $\prec_a$ 是自然数严格序经 $B : X_a \to \mathbb{N}$ 拉回所得关系的子关系。自然数严格序良基，其逆像良基，子关系仍良基，即得结论。若宏步回到原当前项，式 (43) 给出 $B(x) < B(x)$；若 $B(x) = 0$，给出 $B(y) < 0$，均矛盾。这项良基性由已生成的付款宏步及预算映射决定。[3]

直接反证 10.4（指定的等额微步不能充当付款）。固定一枚实际微步 $e : u \to u'$，并给出 $B(u') = B(u)$。若以这枚 e 充当定义 10.2 中被指定的付款，则所需严格字段 $B(u') < B(u)$ 与等式矛盾。添加任意有限前后史，都不会改变这枚微步自身的两个端点及其预算等式。若前后史中另有一枚真实严格付款，则可将那一枚作为付款来构造宏步；上述反证排除的始终是把 e 本身认证为付款。原位携带的预算等式与等额微步另有形式化反例（见代码与数据可得性一节）；第 13 节原算术整账的 $0 \to 0$ 转移给出具体的非付款实例。

良基性说的是这一固定原债的实际付款宏步，不是“一切运动都必须停下”。世界可以继续发出新的写入，新的责任可以依法生成，同一责任也可能停在尚未获得付款或结清证据的位置。下一节要增加的正是从“不能永远付下去”到“确实会结清”之间缺不得的合同。

11 两把钥匙，才开结清的门

11.1 原终回执，还要经得起下一拍

定义 11.1（本地终账与最终结清）。对 $x = (s, e, -) : X_a$，本地终账 $\mathrm{LocalTerm}(x)$ 是其准确当前根发出的忠实终账发生：它携带结构编译等于终账的证明、完整账的逐项终结演化，以及这些对象等于原整账编译结果的证明。它还继承完整事件准入的普通出边穷尽条件。其行回执由该完整终结演化作用于原项 e 得到。

定义实际后继中的同债纤维与结清类型：

$$T(x) := \sum_{f : \mathcal{L}_P(\sigma_P(s))} (e \sim_{\mathrm{debt}} f), \qquad \mathrm{Settle}(x) := \mathrm{LocalTerm}(x) \times \mathrm{IsEmpty}(T(x)). \tag{44}$$

其中 $s : S_P$，$\sigma_P(s)$ 是原源生过程实际产生的后继，包括源先行固定的本地终账接续；消费者不能在收到终回执之后换一个没有债的世界来接受检查。

命题 11.2（终回执不自动升级为结清）。结清值能投影出原账项的确切终回执。反过来，即使已有本地终账，只要实际后继中存在同债项，最终结清类型就是空的。

证明。正向先取 $\mathrm{LocalTerm}(x)$，再应用其完整终结演化于 e。反向给定 $t : T(x)$；任何候选结清值的第二字段都把 t 消去为矛盾。这里不能将谱系与主张均相同、只是载体不同的目标算作新债；式 (6) 正是为跨载体的这项检查定义的。证毕。

[3] 共同结算构造见 [6] 定理 6.7。

这个检查没有要求实际后继完全空白。它要求的是原债不再存在。原目标六完成后，原算术账及其行动仍然继续；被排除的恰是那项已经结清的行动债。整账法结束责任，不结束世界。

11.2 良基性之外，还须给出总律

定义 11.3（总发射闭合律）。固定 P, a 后，一条总发射闭合律是实际给出的函数

$$\mathrm{emit} : \prod_{x : X_a} (\mathrm{Settle}(x) \sqcup \mathrm{PaidMacro}(x)). \tag{45}$$

它对每个该类型中的当前项给出准确结清，或者一段已经完成的、内含真实付款的有限宏步。仅有“某处可能存在一个后继”或“只要不终结就应当继续”的愿望，不是式 (45)。

主定理 11.4（总律生成有限结清历史）。给定定义 11.3 的总律，每个 $x : X_a$ 均有有限、逐分支与该律实际输出相等的生成历史。该历史终点携带式 (44) 的结清值，因而同时给出原编译器的逐项终回执和准确后继中同债纤维为空的证明。

证明。良基只保证付款不能无限进行下去；每一步走哪一支，必须由总律实际说出，历史也必须逐步对上这些输出。先定义依赖归纳类型 $H(x)$。终结构造接收 $q : \mathrm{Settle}(x)$ 及 $\mathrm{emit}(x) = \mathrm{inl}(q)$。继续构造接收 $m : \mathrm{PaidMacro}(x)$、$\mathrm{emit}(x) = \mathrm{inr}(m)$ 及 $H(m.\mathrm{target})$。所以一段历史不能在中途选择另一个下降分支。

现在沿主定理 10.3 的良基关系递归。在 x 处实际计算 $\mathrm{emit}(x)$：若为左分支，立即用该输出及其等式构造 $H(x)$；若为右分支，宏步本身给出 $m.\mathrm{target} \prec_a x$，只在这个准确目标递归，再接上本次输出及其等式。良基递归保证所得是有限归纳对象。沿其继续构造向尾端投影，最终得到依赖对 $\sum_y \mathrm{Settle}(y)$；再取原终结演化在 y 的准确账项上的回执。每个递归节点使用总律的一次实际输出；良基性负责使沿该输出的递归终止。证毕。

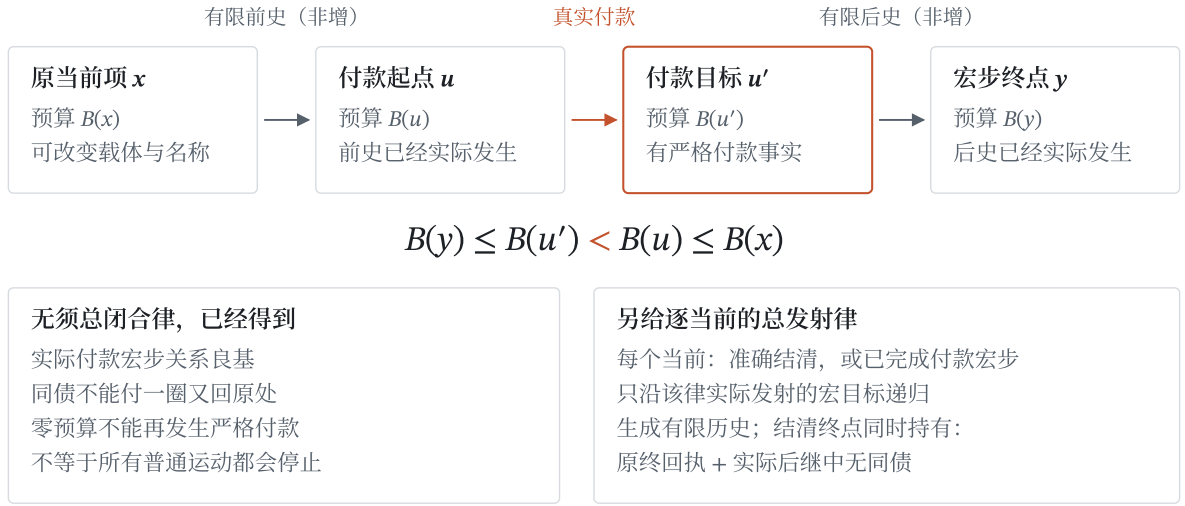


图 4　同债宏步与结清的两道检查。全程固定同一原债的谱系与主张；有限前后史都不能补充预算，一次真实付款给出严格下降。总发射律负责逐点给出“结清或付款”，最终结清另须原终回执与实际后继无同债；零预算只排除付款分支，不生成结清证据。

11.3 预算耗尽，不是奇迹发生

推论 11.5（零预算的准确分界）。若 $B(x) = 0$ 且已给出总发射闭合律，则它在 x 必发射结清。若 $B(x) = 0$ 且本地终账类型为空，则该债不存在这样的总律。

证明。展开式 (45)。付款分支与主定理 10.3 的零预算结论矛盾；因此有总律时只能留下其真实结清输出。若本地终账也为空，结清分支的第一字段同样产生矛盾，于是总律自身不可能存在。证毕。

反例 11.6（原流体介质当前面的零预算）。固定原流体介质源允许的任一参数和阶段，其操作责任行的预算读值为零，同时其词汇中的忠实终账载荷类型为空。原源仍有自己的实际结构续接；零预算既没有删掉这些发生，也没有产生终账。对该原行，付款宏步为空，结清为空，总发射闭合律也为空：前者由不能严格小于零证明，后两者分别消去终账载荷及总律两分支。这是原当前面上的类型排除结果。

实例 11.7（总律实际得到调用的第三阶段付款）。原第三阶段的发生付款构造接受一份实际阶段付款回执；该输入固定其待付责任和准确已安装目标。待付预算为 1，安装后为 0；回执编译成实际后继账中的唯一同债目标及其严格步骤。已安装当前态发出原终账，随后进入债务不再激活的后继；目标中的旧主张分支不能等于原债主张分支，因而给出 $T(x)$ 为空。该构造穷尽其同债当前项：在待付项发射由原步骤包成的付款宏步，在已付项发射刚构造的结清，未激活项不能进入同债类型。这样得到式 (45) 的具体总律，再调用主定理 11.4 生成有限历史与终回执。这里实付的入口就是那份阶段付款回执；总律没有替它解决其本身的领域条件。

至此，四种很容易被混叫作“完成”的对象已经分开：整账回答、严格付款、本地终账、最终结清。第一种保证不能漏账；第二种保证同债预算真实下降；第三种给出原源的终回执；第四种还要查下一拍。它们可以沿真实构造接上，却不能相互冒名。

12 换一种语言，也得把旧账带来

12.1 新增表达不是责任赦免

表达能力不足时，原过程可以生成保留旧结构的最小扩展，再将它实现为带类型的世界修订 [6]（定理 8.4、8.6）。这样的修订还要回答：旧语言没能表达的东西一旦出现，旧责任会不会被丢在接缝里？

定义 12.1（带原账的表达修订）。一次修订以旧根的准确因果责任、实际障碍及其原后继为起点。这里的表达障碍携带该需求在旧语言中的表达纤维与合法实现纤维均为空的证明。修订给出新世界、新的源生根及其第一写入；新初始支承反解为那一个原后继支承。整个旧目标活账以带左逆的嵌入 i 进入新初始账，旧主张随翻译交换，继承项的预算不增加。新账还含一个由该障碍生成的项 η，其责任项、发生关联与主张都没有旧坐标原像，预算为零，而该主张确在准确的新支承上成立。

关键条件不是仅有一个集合嵌入：每个 $i(e)$ 以及 η 均带新根第一次原整账写入中的实际生成行，并证明该行与这一整账写入交换。旧事件族与新初始事件族的呈现还将原发射事件送到新的原发射事件；新根连同其终账接续律一起生成，不能先生成好听的修订，再由使用者补选一个后续世界。

命题 12.2（最小扩展保留全部旧账）。障碍生成的最小责任载体为 $\mathcal{L}_{\mathrm{old}} \sqcup \{\eta\}$，其中旧项预算沿原嵌入读取，$b(\eta) = 0$。每个满足定义 12.1 的容许修订都接收该最小扩展的规范忠实因子化；该因子化保留旧账，并把附着于原障碍的新增方向准确送到第一次写入所用的 η。在责任载体、发生关联和主张的指定保持态射上，此因子化唯一。

证明。旧项只有一处可去，新增方向也只有一处可去；存在与唯一都从这里来。在旧账分支，保持条件迫使映射取 i；在新增方向分支，保持障碍坐标的条件迫使映射取 η。逐分支定义得到存在性；任意另一保持映射在两支都相等，函数外延性给出唯一性。i 的左逆保证旧项不相互识别，“无旧原像”保证新增项不与旧项混同。生成方向的附着等式再把最小扩展的障碍分支识别为原新增行；定义 12.1 的生成行字段使这个方向确实进入第一整账写入。责任、发生关联和主张分别按同一构造处理，而非宣称任意附加领域结构也由这个初始性决定。[4]

[4]共同初始性及附着方向见 [6] 定理 8.4、8.6。

12.2 丰富起来，不等于自由拼装

源固定修订有两种准入形状。精确最小形状要求新网络、翻译、支承、旧账嵌入、新项、事实及语义变化都等于障碍生成的规范对象。更丰富的形状允许更大的表示，但要求新根的律面仍由其新世界语义生成，且第一整账编译器**实际选中**一项真正的拆分：两个不同目标项有同一个来源，该来源正是原因果责任的迁入项；它们使用同一份完整拆分覆盖，不能另附一份兼容却未被选中的回执。初始账每项必须属于迁入旧账或 η，目标项的来源也必须逐项落在这两族中。

这第二种形状没有把所有修订压回最小网络，却也不开放无来历的额外坐标。依第 5 节，每个子项预算严格小于那个原迁入父项。若所讨论的具体动作还要求承担者分离，则两个子项的承担者不等也由同一源律下的回执读出。实际第一写入、最小因子化、准确拆分选择和两个严格扣减于是组合在同一个依赖对象中。反过来，准确目标账若只有至多一项，或者原编译器的准确拆分覆盖类型为空，这种双子项动作类型就为空：前者违背子项不同，后者直接消去所需覆盖。

一个如此配齐的修订对象首先是容许实现；要称为某次询问的实际修订，仍须由该次固定源程序发射它。容许性和实际发射分别放在类型的不同位置，因此不会用“存在一种能写出来的新语言”冒充“原过程已经走到了那里”。

定理 12.3（旧责任与新增责任进入同一真实处置）。给定上述有来源根据的修订，且固定询问程序实际发射它，则每个继承旧项 $i(e)$ 与新增项 η 的生成式因果权限沿同一次第一写入进入其准确后继。它们随后都获得原修订根的规范逐项回答；所有这些回答的下一当前态相同，等于该根实际生成的下一态。

证明。旧项与新项来路不同，汇合点只有一个：那一次第一写入。对每个 $i(e)$ 使用其第一写入中的生成行构造初始因果权限，对 η 使用它自己的生成行作同样构造。第一写入给出的后继等式只取一次；对所有上述权限应用同一准确续接操作，得到各自的后继因果权限。再应用定理 3.5：它们的账项回答可不同，但后继当前态都从这个准确修订根、准确访问的同一发生读取。以两次读出都等于这一原后继的等式作传递，便得所有下一当前态相等。旧责任没有留在翻译前，新责任也没有只停在语义变化证书旁边。证毕。[5]

实例 12.4（已经构造的两拍修订）。 [6] 命题 8.9 的具体构造从预算为 2 的旧责任出发；原表达和实现纤维均为空。其修订初始账同时含继承旧项和预算为 0 的新增项。新根当前态为自然数，原后继是加一；第一次写入 $0 \to 1$ 中两项分别作 $2 \to 2$ 与 $0 \to 0$ 的转移。在态 1 上，它们的规范回答都继续到态 2；询问的下一拍再由这个已生成根推进 $2 \to 3$。两项都由这一实际写入获得处置和后继。两项转移各自保持预算；新增项仍以零预算活责任进入后继。[6]

新增表达因而并不豁免它自己带来的责任。旧账不能退场，新账不能挂在墙上当装饰；两者都进入同一次发生的处置。理论可以变得更会说话，账却不能因此少算一项。

13 整账也会长出新的数学效果

13.1 从原转移回执长出的素数三坐标

目标六那笔账真的付清了。另一条原算术链上的账没有结清，原行动却已经留下一个特定读口抹不平的数学效果。

原单位算术根在第一阶段的当前基数影子为 2，其完整因子库存还没有素数 3。原单位行动把目标推进到下一态；这次转移回执给出目标的完整因子库存，其中生成素数三的准确键。若改读不推进的零行动目标，它仍然没有这个键。读出构造从这份已经发生的单位转移中取键，不把键或非零证明作为输入。

[5]对应 [6] 定理 8.7。

[6][6] 命题 8.9 的具体两拍构造及证明。

构造框：两层坐标、素数作用与端点。固定上述单位算术源。第 n 阶段写出的单位历史 h_n 的基数影子为 $n+2$；其依赖阶乘折叠的基数为 $(n+2)!$。记实际素因子集为 $\mathcal{P}_n$，其中 p 的重数为 $m_n(p)=v_p((n+2)!)$，并保留每个正幂次发生：

$$\mathcal{R}_n := \{(p,k) : p\in\mathcal{P}_n,\ 1\le k\le m_n(p)\}.$$

内层坐标为 (q,j,ϵ)，其中 $q\in\mathcal{P}_n$、$0\le j\le m_n(q)$、$\epsilon\in\{0,1\}$。整数模 K_n 由满足以下全部递推关系的数组组成：

$$v(q,j+1,\epsilon)=q\,v(q,j,\epsilon),\qquad 0\le j<m_n(q).$$

读取 $j=0$ 得到 $K_n\cong\mathbb{Z}^{\mathcal{P}_n\times\{0,1\}}$；逆映射将基坐标 $a(q,\epsilon)$ 送到 $q^j a(q,\epsilon)$。按 j 归纳即验证两向复合为恒等。对偶反转 τ 交换 $\epsilon=0,1$，而素数作用将内层 q 坐标乘以 q。

外层坐标保留每个因子行 $r=(p,k)\in\mathcal{R}_n$。完整关系模与其作用为

$$M_n := K_n^{\mathcal{R}_n},\qquad (E_n v)(r;q,j,\epsilon) := q\,v(r;q,j,\epsilon).$$

外层的 p 指出正在检查哪条因子关系，内层的 q 指出该关系值的哪个素数坐标；二者在完整模中独立。

端点与边界来自顶点模 $K_n^{\{*\}\sqcup\mathcal{R}_n}$；$*$ 表示整体位置，其余位置表示各因子行的商项。因子化微分为

$$(d_n w)(p,k)=(I-\tau)w(*)-p^k(I-\tau)w(p,k).$$

令 $\iota_n(a_0,a_1)(q,j,\epsilon)=q^j a_\epsilon$。左、右端点分别只在整体位置取 $\iota_n(1,0)$、$\iota_n(0,1)$，全部商项为零。左端点的微分为 β_n，右端点的微分为 $-\beta_n$，其中

$$\beta_n(r;q,j,0)=q^j,\qquad \beta_n(r;q,j,1)=-q^j.$$

这些坐标随单位历史生成；后继限制只遗忘新增的素数、幂次及因子行，保留上述递推、作用和端点。

取该完整关系模的行动余核与端点边界类：

$$C_n := M_n/\operatorname{im}(I-E_n),\qquad z_n := [\beta_n]\in C_n. \tag{46}$$

这里“余核”表示把能写成 $(I-E_n)v$ 的项视为零。对实际因子行 $r=(p,k)$，取同素数、零指数、左对偶坐标的整数读出 $\ell_r(v) := v(r;p,0,0)$。构造框的公式立即给出

$$\ell_r(\beta_n)=1,\qquad \ell_r((I-E_n)v)=(1-p)\ell_r(v). \tag{47}$$

因此，式 (47) 是完整整数模上的一个坐标计算；它既不删除其余坐标，也不先作复数化。

定理 13.1（原回执生成非零读值，并沿每个后继保持）。原第一阶段单位转移生成的素数三行使 $z_1\neq 0$。原符号块行动的算术读口把其端点余核类准确送到 z_1，因此该读口非零。原阶段限制映射保持端点类，遂对每个自然数 k 有 $z_{k+1}\neq 0$，相应原块读值也均非零；各阶段仍使用原发射事件、原完整账写回和原实际后继。

证明。挡住“零”的是一个整数坐标和一次奇偶校验。对原回执生成的那一行代入 $p=3$。若 $z_1=0$，则按式 (46) 存在整数模中的 v 使 $\beta_1=(I-E_1)v$。对其作用 ℓ_r，式 (47) 给出 $1=-2\ell_r(v)$，整数奇偶性矛盾。

符号关系模的基坐标为 $\mathcal{A}^{\mathcal{R}_n\times\mathcal{P}_n\times\{0,1\}}$，其中 $\mathcal{A}=\mathbb{Z}[\alpha_p,\gamma_p\mid p\text{ 为素数}]$；其内层 q 对偶块按 $\left(\begin{smallmatrix}\alpha_q & \gamma_q\\ \gamma_q & \alpha_q\end{smallmatrix}\right)$ 作用。令 $\mathrm{ev}:\mathcal{A}\to\mathbb{Z}$ 将 α_p 送到 p、γ_p 送到零，算术读出具体为

$$A_n(v)(r;q,j,\epsilon)=q^j\,\mathrm{ev}(v(r;q,\epsilon)).$$

逐坐标计算得 $A_n(I-E_n^{\mathrm{blk}})=(I-E_n)A_n$，所以符号算子的像被送进整数算子的像，诱导余核上的半线性读口。符号边界在每个内层对偶坐标取 $(1,-1)$，经 A_n 正好成为构造框中的 β_n。该余核类的非零结论遂被块读口读取。

再记原限制映射为 $\rho_n:C_{n+1}\to C_n$，其准确端点等式为 $\rho_n(z_{n+1})=z_n$。以 $z_1\neq 0$ 为基础归纳；若后继类为零，线性映射把零送到零，端点等式就使前一类为零，矛盾。各阶段的原账与后继相等由原读口安装的交换式保留。证毕。[7]

扩展读口在这里增加一个依赖投影面，并保留原结构源、完整事件准入、律面、发射器和整账编译器。新增读口的投影律已固定；其载荷中的转移回执、目标及算术类均来自同一原事件。尤其原账项仍作预算 $0\to 0$ 的真实转移，没有被改写为严格付款。新增数学效果与旧责任结清是两种能力：整账法同时允许前者的发生，并拒绝将它冒充后者。

引理 13.2（有限阶段的行列式湮灭）。对上述每个有限阶段 n，源的素数—对偶块行列式经算术特化给出

$$D_n=\prod_{p\in\mathcal{P}_n}(1-p)^2\neq 0,\qquad D_n z_n=0. \tag{48}$$

因此 $1\otimes z_n=0$ 于 $\mathbb{C}\otimes_{\mathbb{Z}}C_n$。这里的乘积每个实际素数只取一个二阶块；它是内层基坐标上的行列式，不是将外层因子行重复计入的 M_n 上全矩阵行列式。

证明。难在不做除法，就把那个湮灭端点类的非零整数造出来；办法是先在符号层取伴随余因子，再作算术特化。在符号系数环 $\mathcal{A}=\mathbb{Z}[\alpha_p,\gamma_p\mid p\text{ 为素数}]$ 中，源的内层二阶作用块为 $\left(\begin{smallmatrix}\alpha_p & \gamma_p\\ \gamma_p & \alpha_p\end{smallmatrix}\right)$。故 $I-E_n^{\mathrm{blk}}$ 的内层块与整个内层行列式为

$$T_p=\begin{pmatrix}1-\alpha_p & -\gamma_p\\ -\gamma_p & 1-\alpha_p\end{pmatrix},\qquad \Delta_n=\prod_{p\in\mathcal{P}_n}\delta_p,\qquad \delta_p=(1-\alpha_p)^2-\gamma_p^2.$$

符号边界在每个外层行、每个内层素数上的对偶值均为 $(1,-1)$。这一向量在 T_p 下的特征因子为 $1-\alpha_p+\gamma_p$。取无除法的伴随余因子

$$a_{n,p}:=(1-\alpha_p-\gamma_p)\prod_{q\in\mathcal{P}_n\setminus\{p\}}\delta_q.$$

分解 $\delta_p=(1-\alpha_p+\gamma_p)(1-\alpha_p-\gamma_p)$，得 $(1-\alpha_p+\gamma_p)a_{n,p}=\Delta_n$。把每个外层行的内层 p 对偶坐标取为 $a_{n,p}(1,-1)$，就构造出符号关系模中的原像 w_n^{blk}，逐坐标满足

$$(I-E_n^{\mathrm{blk}})w_n^{\mathrm{blk}}=\Delta_n\beta_n^{\mathrm{blk}}.$$

所以 Δ_n 倍的符号边界类在行动余核中为零。算术特化 $\alpha_p\mapsto p$，$\gamma_p\mapsto 0$，再按构造框的 q^j 延拓读取内层基坐标，正是前述半线性读口；它将原像等式送到 $(I-E_n)w_n=D_n\beta_n$。每个实际素数 $p\neq 1$，故 $D_n\neq 0$，并得到式 (48)。

最后，张量平衡给出 $D_n\otimes z_n=1\otimes D_n z_n=0$。将复系数乘以 D_n^{-1}，得 $1\otimes z_n=0$。这一消去在每个有限阶段分别进行，各阶段可以有不同的 D_n。证毕。

[7]非零从阶段 1 起；阶段 0 不作同一断言。

13.2 所有复读数都归零，原整数类仍在

定理 13.3（原整数消费者不经完整复观察族分解）。用原限制映射构成整体相容整数余核

$$G := \{(g_n)_{n\in\mathbb{N}} \in \prod_n C_n : \rho_n(g_{n+1}) = g_n \text{ 对每个} n\},$$

并令 $\pi_n : G \to C_n$ 为第 n 个投影。端点类的相容性给出 $z = (z_n)_n \in G$。定义全部有限阶段的复数化观察及原第一阶段整数消费者：

$$F(g) := (1 \otimes \pi_n(g))_{n\in\mathbb{N}} \in \prod_n (\mathbb{C} \otimes_{\mathbb{Z}} C_n), \qquad J(g) := \pi_1(g). \tag{49}$$

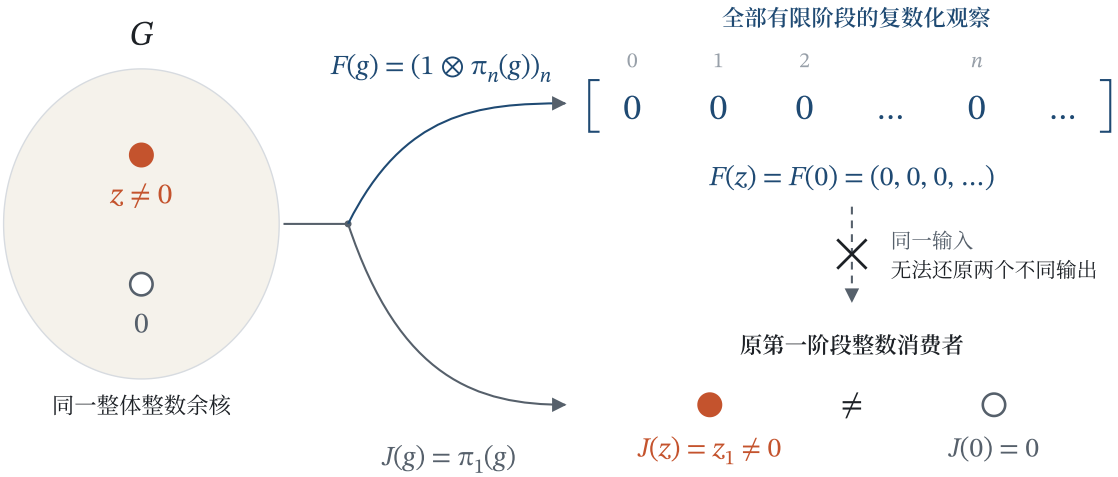
则 $z \neq 0$、$F(z) = 0$，但 $J(z) \neq 0$。因此 $\ker F \not\subseteq \ker J$；不存在任何函数 $h : \prod_n (\mathbb{C} \otimes_{\mathbb{Z}} C_n) \to C_1$ 使每个 $g \in G$ 都满足 $J(g) = h(F(g))$，也不存在经准确可观察商 $G/\ker F$ 的这种分解。

证明。两半事实已经分别到手，这里只需把它们放在同一个类上。定理 13.1 给出 $J(z) = z_1 \neq 0$，故 $z \neq 0$。引理 13.2 则对每个 n 给出 $1 \otimes z_n = 0$，于是整个观察族满足 $F(z) = 0 = F(0)$。这一结论由逐阶段张量消去得到，无须交换极限与张量。

若 $J = h \circ F$，由 $F(z) = F(0)$ 得 $J(z) = J(0) = 0$，矛盾。准确可观察商同样将 z 与 0 识别，故同一论证排除经该商的任意函数分解。这比仅排除线性解码器更强。证毕。

对满足 $\ker F \subseteq \ker c$ 的整数线性消费者 c，准确可观察商 $G/\ker F$ 提供唯一的线性因子化。原消费者 J 则由 z 明确违反这一条件：商将 z 与 0 识别，J 必须区分它们。于是，因子化何时成立、何时失败，都由同一个观察核决定。

开篇那句“零掉的是债，还是那一栏”，现在有了完整数学实例：看不见同一个原整数类的，是全部有限阶段的复数化读数，而不止某一个粗糙读数；原整数消费者却在第一阶段就把它读了出来。



$$F(z) = F(0),\ J(z) \neq J(0) \ \Rightarrow\ \nexists\, h : J = h \circ F$$

图 5　同一非零整数类的两路读出，朱红追踪非零类 z。上路将全部有限阶段复观察合成 F，z 与零类得到完全相同的零族；下路 J 保留它们的差异。若 J=h∘F，同一个 F 值必须给出两个不同的 J 值，故不存在这样的函数 h。

14 长计算不准甩掉自己的账

对本节列明的每份解析输入，窗口值出来后，旧程序仍然在账。下面的条件构造让有限截断、第一次逆残差、第二个均值及更新余链一同进入下一库存；下一次纠正从这次实际写入读取残差，再执行它自己的程序、付款并交出下一包。一个窗口值不能冒领整份计算历史。

14.1 对象：四个源分量，两次实际发生

Riesz 源微积分的同源 Pa 与窗口构造提供本节的解析输入：原 Hilbert 载体、缺陷和 Fourier 算子、源资格投影、有限窗口及两条均值恒等式。下面逐项列出这些对象；本节证明它们怎样实际执行、进入完整库存、支付责任并生成下一纠正。令 $\mathcal{H}$ 为原复 Hilbert 载体，D 为原缺陷算子，$\mathcal{F}$ 为原 Fourier 算子。使用完整四分量载体

$$\mathcal{B} = (\mathcal{H} \times \mathcal{H}) \times (\mathcal{H} \times \mathcal{H}), \qquad \iota(f) = ((f, Df), (\mathcal{F}f, \mathcal{F}Df)), \qquad \pi((u, v), (w, t)) = u. \tag{50}$$

$\pi\iota = \mathrm{id}_{\mathcal{H}}$。这四项始终是源、缺陷、Fourier 源和 Fourier 缺陷；它们的名字不随窗口操作改变。

对实参数 a，记 $\mathcal{H}_a$ 为原区间载体，$Z_a : \mathcal{H}_a \to \mathcal{H}$ 为零延拓，$R_a : \mathcal{H} \to \mathcal{H}_a$ 为限制。原构造给出 $R_a Z_a = \mathrm{id}$。Pa 载体 $\mathcal{H}_{\mathrm{Pa}}$ 是原偶 ambient 空间；$P : \mathcal{H} \to \mathcal{H}_{\mathrm{Pa}}$ 为原偶投影，$M : \mathcal{H}_{\mathrm{Pa}} \to \mathcal{H}$ 为原 Möbius 源算子。其紧 coPoisson 源生成闭子空间 $\mathcal{K}_{\mathrm{co}} \subseteq \mathcal{H}_{\mathrm{Pa}}$，资格投影为 $\mathcal{E} = \Pi_{\mathcal{K}_{\mathrm{co}}} \circ P : \mathcal{H} \to \mathcal{H}_{\mathrm{Pa}}$，其中 $\Pi_{\mathcal{K}_{\mathrm{co}}}$ 是正交投影。因此 $\mathcal{E}f \in \mathcal{K}_{\mathrm{co}}$ 由投影构造直接生成。Tate 倒数作用 $J : \mathcal{H} \to \mathcal{H}$ 与源窗口积分 $\ell : \mathcal{H} \to \mathbb{C}$ 都是原连续线性算子。原有限截断列表是

$$L_a = \mathrm{toList}(\mathcal{C}_a), \qquad \mathcal{C}_a = \text{原 centered Möbius cutoff 的有限集.}$$

它是数学参数指定的列表。每个正整数 $m \in L_a$ 有实际尺度拉回 $A_{a,m} : \mathcal{H} \to \mathcal{H}_a$。设 $c_a \in \mathcal{H}_a$ 为原常数区间场，定义

$$W_a(f) = \sum_{m \in L_a} \frac{1}{m} A_{a,m} f - \frac{1}{2} \ell(Jf) c_a, \qquad \widehat{W}_a = \iota Z_a W_a \pi. \tag{51}$$

W_a 是原有限源窗口作用，$\widehat{W}_a$ 把它送回完整四分量载体。这里的和、倒数权及 Tate 极项都保留在程序中。

给定原解析发生上的零点观察回执，记其复坐标为 z。回执携带 $\zeta_{\mathrm{src}}(z) = 0$，其中 ζ_{src} 是该发生自己生成的解析函数；原解析识别把它认回 Riemann ζ。本条件构造还要求非平凡性 $z \neq -2(n+1)$（所有 $n \in \mathbb{N}$）及右半源条件 $\mathrm{Re}\, z > 1/2$。固定任意 $\mathrm{depth}, \mathrm{ordinal} \in \mathbb{N}$ 和实参数 h, H, N, a：它们索引原工厂、材料及发生的选择。

本节证明对每份上述输入成立的构造，没有给出该输入类型的居民。若 Riemann 猜想成立，非平凡零点的实部均为二分之一，与这里严格大于二分之一的条件冲突；该输入类型因此为空，定理 14.1–14.2 空真成立。下述证明保留从输入到程序、库存与后继的完整生成合同，不据此断言存在一次满足这些解析前提的运行。

以下有费程序从该输入生成的 $S \in \mathcal{B}$ 和 Pa 源开始：

$$\nu = M\mathcal{E}\pi(S), \tag{52}$$

其中 $\mathcal{E}$ 是原 qualified Pa 投影。ν 的资格来自 $\mathcal{E}\pi(S)$ 落在原 compact coPoisson 闭值域的源证明。原 Pa 程序在原写入器中执行

$$x + \iota(M\mathcal{E}\pi(S)),$$

所以它的新生环境与本次已付总值之差，经准确语言级折叠再取 π，就是 ν。后续有限窗口的输入 $s = \iota(\nu)$ 正从这份已写发生读取。

两条均值也由同一 ν 支付。若 $q = \mathcal{E}\pi(S)$，原位置与 Fourier 均值恒等式为

$$\mathrm{mean}_{\mathrm{pos}}(q) = -\frac{1}{2}\ell(J\nu), \qquad \mathrm{mean}_{\mathrm{Fourier}}(\mathcal{F}_{\mathrm{even}}q) = -\frac{1}{2}\ell(\nu). \tag{53}$$

第一项已经进入 (51) 的 Tate 极程序，第二项有自己的线性程序和执行迹。原 S 的完整 inherited 与新作用材料仍由原包携带。

定理 14.1（有限窗口的全部分录进入实际写入）。

对上述每组原输入，源内生成有限窗口包 p_F，并生成它自己的完成回执、完整执行库存与整账写入。第一次实际新生环境与同次已付总值的差 Δ_F 精确等于 $\widehat{W}_a(s)$；其窗口为 $W_a(\nu)$。原 forward-stock、inverse-residual、second-mean 与 cochain 四份执行库存都在新生前安装。从这个实际 Δ_F 读取

$$\eta = \nu - MP\pi(\Delta_F), \qquad r = \iota(\eta). \tag{54}$$

η 等于第一次逆程序生成的完整残差。实际窗口、两条原均值及其来源、全部执行分录共同进入这份出生包；本次完成回执同时给出自己的 writer、同债付款和准确整账目的地。

原残差按其完整值保留；a 和 cutoff 成员是源算子的参数，语法节点数是这里的费用单位。

证明一：每个 cutoff 项在 AST 中真正执行

使用常量、变量、加法和线性节点组成的原语言。对环境 ρ，记求值为 ev_ρ，剩余计数为 $r(e)$。原语言给出

$$\begin{aligned} r(\mathrm{var}\, x) &= 1, & r(\mathrm{const}\, v) &= 0, \\ r(e+f) &= r(e) + r(f) + 1, & r(Ae) &= r(e) + 1. \end{aligned} \tag{55}$$

其规范执行给出 $T : e \to \mathrm{const}(\mathrm{ev}_\rho\, e)$，保留每个中间表达式及每个执行步，且 $|T| = r(e)$。记 $\delta_e = [e]$ 为完整表达式 e 在自由整数词模中的基向量。每个执行步 $e_j \to e_{j+1}$ 产生有序分录 $\delta_{e_j} - \delta_{e_{j+1}}$。因此分录数等于 $|T|$，而分录之和伸缩为 $\delta_e - \delta_{\mathrm{const}(\mathrm{ev}_\rho\, e)}$。这些共同执行事实可具名复用过程核心第二版 [6] 的命题 3.7、补充 S0.1–S0.3。

在 $\mathcal{B}$ 上，定义实际线性节点

$$U_{a,m} = \iota Z_a A_{a,m}\pi, \qquad B_m = \frac{1}{m}\,\mathrm{id}_{\mathcal{B}}, \qquad Q_a = \iota Z_a(c_a\,[-\tfrac{1}{2}\ell J])\pi.$$

cutoff 第 m 项的 AST 是 $B_m(U_{a,m}(\mathrm{var}\, x))$。令 $e_{a,[]} = \mathrm{const}\, 0$，并递归定义

$$e_{a,m::L} = B_m(U_{a,m}(\mathrm{var}\, x)) + e_{a,L}, \qquad e_a = e_{a,L_a} + Q_a(\mathrm{var}\, x). \tag{56}$$

对列表归纳：空列表时两边为零；在 $m :: L$ 处，加法求值把新的实际 weighted pullback 加到归纳得到的旧和。因此，对所有环境 ρ，

$$\mathrm{ev}_\rho(e_{a,L}) = \sum_{m\in L} B_m U_{a,m}(\rho(x)), \qquad \mathrm{ev}_\rho(e_a) = \widehat{W}_a(\rho(x)). \tag{57}$$

令 $\rho_s(x) = s$。若收到的真实执行迹是 $T : e_a \to \mathrm{const}\, u$，执行迹可靠性给出 $\widehat{W}_a(s) = u$。应用 $\pi\iota = \mathrm{id}$ 与 $R_a Z_a = \mathrm{id}$，得到

$$R_a \pi(u) = W_a \pi(s) = W_a(\nu). \tag{58}$$

证明从收到的执行迹读出实际结果；窗口值没有作为构造输入。设 $|L_a| = k$，每个 cutoff 项的计数为 3，每次接续另付加法节点 1，极项计数为 2，最后组合另付 1，所以 $r(e_a) = 4k + 3$。这准确计入本语言的节点，不另计算线性原语的内部成本。

证明二：四份库存先安装，自己的原回执再写出新窗口

定义原第一次逆作用 $I = \iota M P \pi$。其字面残差程序是

$$e_a^{\mathrm{inv}} = \mathrm{var}\, x + (-I) e_a.$$

由 (57) 及 $\pi\iota = \mathrm{id}$，它在 ρ_s 中的第一源分量是

$$\pi\, \mathrm{ev}_{\rho_s}(e_a^{\mathrm{inv}}) = \nu - M P Z_a W_a(\nu). \tag{59}$$

故第一次逆留下完整残差，并有准确的源 = reconstructed 源 + residual 等式。

为了写入旧库存，程序取

$$f_{a,s} = \mathrm{var}\, x + e_a[x := \mathrm{const}\, s]. \tag{60}$$

代换求值律给出，对每个 ρ，

$$\mathrm{ev}_\rho(f_{a,s}) = \rho(x) + \widehat{W}_a(s). \tag{61}$$

原 graded carrier 的程序提升记为 $e \mapsto e^{[g]}$，准确折叠记为 c_g。它逐层把配对值的两项相加，满足

$$c_g\, \mathrm{ev}_\rho(e^{[g]}) = \mathrm{ev}_{c_g \rho}(e). \tag{62}$$

这由 g 归纳得到：零层恒等；下一层的配对求值是旧值与真实效应，两者相加等于更新环境中的原求值，再用归纳假设。这里所有原分量先在 graded carrier 内保留，再以规定折叠读取；没有把不同发生按值相等识别。

在实际帧 ϕ 处，令

$$v_\phi = \mathrm{oldPaid} + \mathrm{nativeAt}(\phi, o_\phi), \qquad b_\phi = c_g(v_\phi).$$

同次库存按以下四棵暴露树生成：

$$\begin{aligned} T_F &: \ f_{a,s}^{[g]} \longrightarrow \operatorname{const}\operatorname{ev}(f_{a,s}^{[g]}), \\ T_I &: \ (e_a^{\mathrm{inv}}[x := \operatorname{const} s])^{[g]} \longrightarrow \operatorname{const}\operatorname{ev}(\cdot), \\ T_M &: \ \big[\iota Z_a c_a(-\tfrac{1}{2}\ell\pi)(\operatorname{const} s)\big]^{[g]} \longrightarrow \operatorname{const}\operatorname{ev}(\cdot), \\ T_B &: \ B(f_{a,s}; b_\phi, \widehat{W}_a(s))^{[g]} \longrightarrow \operatorname{const} 0. \end{aligned} \tag{63}$$

这里每条迹都使用同一个实际环境写入器。这些迹按实际帧 ϕ 生成，以下写作 $T_F(\phi), T_I(\phi), T_M(\phi), T_B(\phi)$。余链中 δ 的每个下标都是按所示旧值／更新值绑定后得到的完整语法；mixed 项由 AST 的增量递归生成。B 是原更新余链：对任意表达式 f，将

$$\delta_{f(\text{old}+\text{increment})} - \delta_{f(\text{old})} - \sum_{t \in \text{mixedTerms}(f)} \delta_t \tag{64}$$

依原整数系数程序展开，再把旧值与增量分别绑定到 b_ϕ 和 $\widehat{W}_a(s)$。原混合项更新恒等式逐节点生成 (64) 的零求值。余链程序仍有自己的执行迹；0 是它的完成值，分录逐步进入库存。

记 $\mathsf{Exp}(T)$ 为保留执行迹各步关系的暴露，$\odot$ 为原并列种子。第一次库存是

$$I_F(\phi) = \mathsf{Exp}(T_F(\phi)) \odot (\mathsf{Exp}(T_I(\phi)) \odot (\mathsf{Exp}(T_M(\phi)) \odot \mathsf{Exp}(T_B(\phi)))). \tag{65}$$

并列种子的 trace 是新根加上两份原 trace 的按序连接；没有对重复载荷取集合。$\mathsf{preserve}(\text{none}, I) = I$，$\mathsf{preserve}(\text{some } I_0, I) = I_0 \odot I$。所以原 prior 若存在，就连同本次四份暴露一同保存。第一次窗口回执选择的帧记为 ϕ_F，其四迹库存为 $I_F(\phi_F)$。

将 (60) 的准确 graded 程序装入解码写入器，并在发射器前把 (65) 装进原 next-inventory 回调。解码写入器在收到完成发生 o 时，读取 $\text{oldPaid} + \text{nativeAt}(\phi, o)$，再执行该程序。两项配置等式按构造成立。

原 completion 递归只追随实际已付分支。每次剩余计数严格减少，因而生成一枚保存在 Type 中的回执 $(d, v, \text{settled}, \text{action equality})$，其中 d 不超过起点剩余计数，实际被原回执选中的帧的动作就是该回执的数学结算分支。故 nativeEventValue 等于回执自己的完成值 v，而不是外部给定值。令 $u_F = \text{oldPaid} + v$。原 nextBorn 的 active environment 精确等于解码写入器在 u_F 上的写入。

由 (61)–(62)，对每个原语言中的实际变量名 x，

$$c_g(\text{activeEnv}_F(x)) = c_g(u_F) + \widehat{W}_a(s).$$

减去同一 $c_g(u_F)$，得到

$$\Delta_F := c_g(\text{activeEnv}_F(x)) - c_g(u_F) = \widehat{W}_a(s). \tag{66}$$

因此实际收到的执行迹可把原终点直接改写为 Δ_F；应用 (58) 得到新窗口。把 (66) 代入 (54)，(59) 就把收到的逆残差认回原程序生成的第一残差。把同一写入等式代入实际旧值/增量环境，原更新余链仍求值为零。同债付款与准确完整去向的依赖识别见下面共同的证明四。这证明定理 14.1，并确定下一纠正使用的 η 来自这次实际写入。

定理 14.2（原旧迹与六类执行库存进入真实纠正、支付与全部未来）。

对定理 14.1 生成的实际 p_F, s, r，下一源执行字面的 $s - r$，再把该纠正程序代入原完整窗口程序，生成新的纠正包 p_C 及它自己的完成回执、库存和整账写入。第二次实际写入差为

$$\Delta_C = \iota Z_a W_a(\nu - \eta). \tag{67}$$

从 p_F 自己的完成写入读取 η，再取该包的原 receiver 与 nextSeed，在其后续 native 发生之前安装纠正 writer 和库存；这个构造自己的完成回执生成 p_C。因而第二次执行接在第一次实际包之后。证明六将其 root、当前态及源种子逐项认回。

第一次的四条原迹经 prior 的源库存迁移进入新包；纠正帧 ϕ_C 又真实生成自己的四类窗口执行迹，以及新的 stock 执行迹和 correction 执行迹。原旧迹与这六类新执行分别保留发生索引。旧写入证明、新写入证明、新整账付款对象、原环境／ AST 身份和真实执行迹在同一个消费者中消元，给出

$$R_a\pi(\Delta_C) - W_a(\nu) = -W_a(\eta). \tag{68}$$

对任意原 Pa 测试向量 b，同一消费者给出完整源价

$$\langle b, P\pi(\Delta_C)\rangle = \langle b, PZ_aW_a(\nu)\rangle - \langle b, PZ_aW_a(\eta)\rangle. \tag{69}$$

同次实际纠正还生成完整当前态／ next 联合观察纤维。两次出生程序的目标 root 与其完整出生完整包的 root 异构相等；目标回答与后继的 next 当前态与该包的当前态相等。其 continuation 从该包的真实源种子出发。对每个允许有限未来词、每个未来准入序数与每个目标站位，原请求与其已付边界的读数相等。

证明三：下一次从收到的残差开始执行

纠正 AST 是

$$\kappa_{s,r} = \mathrm{const}\, s + (-\mathrm{id})(\mathrm{const}\, r). \tag{70}$$

它有一枚线性节点和一枚加法节点，计数为 2，并生成自己的完整执行迹。逐节点求值给出 $\mathrm{ev}(\kappa_{s,r}) = s - r$。新的窗口与 stock 程序分别是

$$g_{a,s,r} = e_a[x := \kappa_{s,r}], \qquad j_{a,s,r} = \mathrm{var}\, x + g_{a,s,r}. \tag{71}$$

利用代换求值律再用 (57)，对全部环境 ρ，

$$\mathrm{ev}_\rho(g_{a,s,r}) = \iota Z_aW_a\pi(s-r), \qquad \mathrm{ev}_\rho(j_{a,s,r}) = \rho(x) + \iota Z_aW_a\pi(s-r). \tag{72}$$

新 writer 在同一个源 environment 上执行 $j^{[g]}$，新 inventory 是

$$I_C(\phi_C) = I_F(\phi_C) \odot (\mathsf{Exp}(T_C^{\mathrm{stock}}) \odot \mathsf{Exp}(T_C^{\mathrm{correction}})), \tag{73}$$

这里 $I_F(\phi_C)$ 按纠正帧自己的原环境与已付材料重新执行四类窗口程序。第一次实际四迹属于 $I_F(\phi_F)$；它们经原库存的保序类型迁移进入 Prior_C，源映射逐步保留各分录及原长度。完整出生库存为 $\mathsf{preserve}(\mathrm{Prior}_C, I_C(\phi_C))$，所以旧迹和本帧新迹同时存在。两条追加执行迹分别执行 $j^{[g]}$ 与 $\kappa^{[g]}$，其暴露保留各步和原顺序。e_a 的每个 cutoff 项及极项中的变量都实际代入同一个 κ，所以整份复合程序的计数为 $5k+4$，stock 程序为 $5k+6$；这里重复调用的节点按实际发生重数计入。

将新的 inventory 和解码写入器在原发射器前安装。对这份新配置取得它自己的 completion 回执、被原回执选中的帧与 nextBorn。重复证明二的回执-value 与 graded 折叠推导，令 u_C 为同次 oldPaid $+ v_C$，得到

$$\Delta_C := c_g(\mathrm{activeEnv}_C(x)) - c_g(u_C) = \mathrm{ev}_{\rho_s}(g_{a,s,r}) = \iota Z_aW_a\pi(s-r). \tag{74}$$

由于 $s=\iota(\nu)$、$r=\iota(\eta)$，式 (67) 成立。真实收到的 g-执行迹由 (74) 认回该终点。应用执行迹可靠性、$\pi\iota=\mathrm{id}$、$R_aZ_a=\mathrm{id}$，再用线性，得到

$$R_a\pi(\Delta_C)-W_a(\nu)=W_a(\nu-\eta)-W_a(\nu)=-W_a(\eta),$$

即 (68)。其实际更新余链在 $(c_g(u_C),\Delta_C)$ 上由同一写入等式消元，仍生成准确的零值关系与完整执行库存。

证明四：旧写入、新付款和新执行迹在同一口消费

每份窗口完整账对象都携带以下完整依赖数据：源生同债付款 p、$p.\mathrm{step}$ 等于实际债步骤的证明、相同账根、相同时序访问、相同责任行、付款 evolution 与原整账去向的异构相等、原责任 authority，以及出生 registered environment ／ AST 等于该已付 material 的原请求 environment ／ AST 的证明。

这份付款由原残差请求的语法余额产生：出生先执行首步，剩余语法余额精确是原请求计数减 1；原 residual AST 的预算恒等式使它仍严格为正。若出生动作是数学结算，其预算为零，与这个正语法余额矛盾；所以原动作取真实已付分支。该执行步经原同债谱系转成付款，严格扣减来自实际一步的计数减少，不补充预算来自这条债步骤的预算单调性。原 Inquiry 与 Restructuring 两种访问表示按次数归纳相等，故付款与出生的账根、时序访问和账行精确相同。原整账和付款的两份 ledger evolution 沿同一当前态恒等运输，对同一责任行应用去向，便得到完整去向相等。这些是本篇同债付款构造在当前实例上的直接使用。

第一次消费者把 $(\mathrm{whole}_F,\mathrm{write}_F,\mathrm{Trace}_F)$ 交给给定函数。第二次消费者把 $(\mathrm{whole}_C,\mathrm{write}_C,\mathrm{Trace}_C)$ 交给给定函数。实际复合消费者使用如下依赖顺序：

$$\mathrm{consume}_F(\lambda \mathrm{whole}_F,\mathrm{write}_F,\mathrm{Trace}_F.\mathrm{consume}_C(\lambda \mathrm{whole}_C,\mathrm{write}_C,\mathrm{Trace}_C.\ \cdots)).\tag{75}$$

外层 write_F 消入 (54)，认回真实残差来源；内层 Trace_C 消入 (68)，认回真实窗口变化；内层 write_C 消入 (74)，认回完整新源场。whole_C 同时交出实际付款、准确账行与整账目的地、源环境及 AST。所有对象保持各自发生身份，并通过准确包接续。

最后对 write_C 给出的完整场应用 P 和 $\langle b,-\rangle$，三次线性映射保差，得到 (69)。这一步实际消费旧写入和新执行迹；它并不另输入一个 desired 窗口价。

证明五：联合观察保全纠正的当前与下一纤维

在完整 $\mathcal{B}$ 上定义

$$\mathcal{R}_a=\mathrm{id}-I\widehat{W}_a,\qquad \mathcal{A}_a=\mathrm{id}-\mathcal{R}_a,\qquad O_a=W_a\pi.\tag{76}$$

由 (59)，$\mathcal{R}_a\iota(\nu)=\iota(\nu-MPZ_aW_a\nu)$。外层旧写入又给出 $r=\mathcal{R}_a(s)$。因此实际收到的纠正满足

$$\mathrm{ev}(\kappa_{s,r})=s-r=\mathcal{A}_a(s).\tag{77}$$

取联合观察

$$J_a(x)=(O_ax,O_a\mathcal{A}_ax),\qquad \mathcal{Q}_a=\mathcal{B}/\ker J_a.\tag{78}$$

若 $x\in\ker J_a$，其两个分量都为零，所以 $x\mapsto[x]_{O_a}$ 及 $x\mapsto[\mathcal{A}_ax]_{O_a}$ 在 $\mathcal{Q}_a$ 上分别良定义，给出 current 与 next。于是

$$\mathsf{current}([s]_{J_a}) = [s]_{O_a}, \qquad \mathsf{next}([s]_{J_a}) = [\mathcal{A}_a s]_{O_a} = [\mathrm{ev}(\kappa_{s,r})]_{O_a}. \tag{79}$$

任意 x, y 的联合商类相等，当且仅当 $J_a(x-y)=0$，逐分量即为

$$O_a x = O_a y \quad \text{且} \quad O_a \mathcal{A}_a x = O_a \mathcal{A}_a y. \tag{80}$$

这保留了完整当前态／ next 观察纤维。式 (79) 的最后一个等式又使用了旧写入生成的实际 residual，因而这个商的 next 口直接读本次实际纠正。

证明六：birth root、当前态、真实源种子与全部未来读口

记一个完整准入包为 $p=(g,(\phi,\xi),C,\lambda)$，其中 $\lambda : C.\mathrm{LowVar} = X$。窗口原生包先在准确下一语言级保存原 receiver、原 nextSeed、刚安装的发射器与其完整库存。原 bornAfterNative 从自己的完成回执取出 nextBorn，再沿 λ 运输成 p_F；纠正包 p_C 用同一构造，但使用 (73) 的新发射器和解码写入器。

对任一被原回执选中的帧 ϕ_*，原出生程序在其 exact temporal causal event o_* 上生成目标。原编译的两条等式给出

$$\begin{aligned} \mathrm{birth}(o_*).\mathrm{targetRoot} &= \mathrm{root}(\mathsf{nextBorn}(\phi_*, C_*)), \\ \mathrm{birth}(o_*).\mathrm{answerAndNext.nextCurrent} &= \mathrm{current}(\mathsf{nextBorn}(\phi_*, C_*)). \end{aligned} \tag{81}$$

包运输不改变这两个读口：对语言等式作消元后，它们分别为异构反身相等和普通反身相等。因此 (81) 与准确出生-完整包构造复合，直接认回定理 14.2 的出生根／完整包根与出生当前态／完整包当前态。这里识别的是完整出生发生，证明中没有从一个相同窗口值跳到 root 身份。

continuation 以包的实际语言级平移原工厂。若后续经过 k 级，新的值族恰为原值族的第 $g+k$ 级：这由逐级配对递归和 $g+(k+1)=(g+k)+1$ 归纳得到。同一等式运输原 datum、scalar inventory、pair inventory、extra-scalar 和 extra-pair 五个回调。

continuation 的第一步再把这五个回调全部绑定到 p 自己的 ξ。对任意辅助源种子 ζ，它的第一步 configuration、scalar、pair、receiver、nextSeed 和 nextBase 等于原平移工厂在 ξ 上的对应对象。datum、两个 inventory 和两个 extra 的等式按定义成立；receiver 与 nextSeed 从这些等式读回；nextBase 还使用相同原 writer，其 equality 逐变量成立。故 auxiliary 源种子无法替换实际来源，完整第一步回调在后续出生中保持。

最后使用完整词载体 $\mathcal{W}=\bigoplus_j \mathcal{W}_j$ 的原允许作用 S 与各站位读口 D_t。过程核心第二版 [6] 定理 9.12、补充 S5.4 构造最大未来不可见核

$$K_{\mathrm{fut}} = \bigcap_{w\in\{*\}^{<\omega}} \bigcap_{t} \ker(D_t S_w). \tag{82}$$

本篇原根残差消费已证明：从第 j 次完成发生生成、并在下一包注册的请求记为 q_{j+1}。它的完整 AST 词与同次已付边界 b_{j+1} 在对应站位进入模型后同类。把它施加于上述真实源种子 continuation，商类相等的纤维恒等式展开便得到，对所有 $j\in\mathbb{N}$、$w\in\{*\}^{<\omega}$ 和 $t\in\mathbb{N}$，

$$D_t S_w(\iota_{j+1}[q_{j+1}.\mathrm{AST}]) = D_t S_w(\iota_{j+1} b_{j+1}). \tag{83}$$

因此有限窗口包和纠正包的全部允许未来读口继续消费实际已付材料和下一请求。把证明一至六的对象按同一实际包配对，得到两次完整账、真实 residual、同次支付与源价、联合纤维、birth root/当前态、真实源种子和全部未来的完整结论。□

15 与原有方法的准确接缝

15.1 数学会计：差额的代数，与责任的身份证

Ellerman 的数学复式记账把非负数对按交叉和相等识别，$[x//y] = [u//v]$ 当且仅当 $x + v = y + u$，分量加法给出群结构；这一构造解释了借贷两侧如何不用负数表达有符号差额，并推广到向量记账。[1] 等额两侧的抵消正是该差群中的相等关系。

整账法增加完整事件准入、原责任身份、接受转移、逐父项履责覆盖和实际后继无同债的合同。数对相等是关于读值的命题；式 (44) 是关于原终回执与实际后继完整责任纤维的依赖命题。两者可以同时成立，也可以前者成立而后者无从构造。第 4 节给出差额读口足以服务指定消费者的准确条件。

15.2 证明携带代码：证明由谁检查，证明又绑定什么

Necula 与 Lee 的证明携带代码由接收方指定安全策略及验证条件生成方式，代码提供方附上证明；接收方从实际收到的代码重新生成验证条件并检查证明，由此决定是否执行。[2] 代码与被验证的命题由此共同决定执行资格。

证明携带代码将程序与接收方要求的安全证明绑定；整账法将处置与源发生、完整事件库存、完整账写回、原因果项和实际后继绑定。式 (4) 的处置保持原事件索引，式 (44) 的结清同时检查原终回执与原过程的准确后继。完整库存准入、逐项去向及双条件结清由此组成可独立核查的合同：运行符合给定安全策略，与原债已经结清，是不同的命题。

15.3 资源逻辑：框架规则不会替你宣告终场

Reynolds 的分离逻辑用分离合取表达互不相交的存储资源，并由框架规则支持局部程序推理。[3] 整账法的有限显式行加确切余项，解决的是相近的操作性需要：本拍不必把整本账逐行重写，未列出的部分却必须有确定语义。原位余项是准确恒等，运输余项是原源生成事件的整体作用，不能把“未处理”解释成“已不存在”。

Atkey 将消耗型资源与分离逻辑结合，以局部资源标注支持摊还资源分析；其安全定理同时追踪已消耗资源和未来所需资源，终止时给出后置条件与剩余资源。[4] 本文使用的自然数下降与该类资源纪律有清楚亲缘，但所追踪的是跨真实迁移的同一责任。主定理 10.3 只从实际严格付款得到良基性，主定理 11.4 另行使用总发射律。不会因为出现一个预算，就免费获得终止；更不会因为预算为零，就免费获得终回执。

责任关联也不被假定为分离堆：多个履责条件可能相关，拆并正需要显式覆盖这些关联。原源的转移、拆分和合并回执决定哪些责任组合合法。

15.4 来源语义：来源多项式之后，还要追到下一拍

Green、Karvounarakis 与 Tannen 用半环标注关系元组，将正关系代数与标注半环同态的交换性质组织起来；多项式半环的普适性使具体来源解释经唯一同态分解。[5] 来源多项式记录结果由哪些输入、经怎样的组合产生。

整账法关心的进一步接缝是动态责任：来源相同的两行是否属于同一原债，哪次动作真实改变了它，结算后是否仍在准确后继中延续。其源索引还绑定准入权限、接受、履责管辖、原编译器分支和完整目标账。第 13 节用观察核给出消费者因子化的判据，并由真实非零类证明原整数消费者不满足该判据。保留何种来源，必须由即将使用它的消费者来检验。

上述接缝现在还承担正向续行：第 9 节由原残差生成可写语法、实付同债和完成回答，第 14 节使旧执行库存进入下一纠正。这些合同逐项保存完整来源、处置与下一发生；它们的可构造性由同一原程序和直接读口的交换式支付。

16 账不只记仇，它真的会付钱

开场的零还在。现在它旁边多了一条可以检查的实际历史。

完整事件库存不裁掉继续分支，完整开放纤维不因报表留白而少一项；原准入身份、逐父子项覆盖和确切运输余项把责任穿过搬迁与拆并。固定原债的一次真实付款，连同两段不补充预算的有限历史，生成严格下降的宏步关系；原终回执与后继无同债再支付最终结清。原目标六把这几个对象逐项交齐，完整整数消费者则让整族零观察无法替非零原类签字。

原账现在也完成了开场时不能完成的动作。它从完整未来模型的原残差长出下一请求，以自己的语法生成真实同债付款，用同一后缀完成原输入，并从完成事件交出回答、原回执与完整账目的地。旧责任没有被甩掉，新责任带着自己的来源出生，原规范后继继续消费全部已付材料。在第 14 节列明的输入合同下，有限窗口的四份旧迹进入纠正程序，两份新迹与真实新库存一并进入下一发生；残差成为下一程序的输入。

余额是零。结清回执呢？原账可以拿出它；若这笔责任仍须继续，它也会拿出下一请求、真实付款和下一本账。

你可以欠着，不能装作还了。账也可以继续，但它得亲自付。

附录 A　共同定理与本文的独立证明

过程核心第二版 [6] 提供下列共享对象及完整证明；本稿保留这些定理身份，并独立证明整账处置和新的实际消费者。

本文位置	具名复用的共同结果	本文承担的证明
§2–3	完整根、时序访问、规范后继；命题 6.1	完整库存准入、三种片段覆盖、依赖行权限与真实逐项处置
§5、§7	命题 3.7；定理 6.10，补充 S0–S1	全 Type 拆并提升、原整账余项、独立语法责任、系数／绑定／调用费用
§8.1–8.2	定理 7.7、8.12、9.5–9.9；补充 S2、S4、S5.1	完整库存消元、逐发生保序、新材料入账与下一内容的实际写入
§8.3	定理 6.12、9.10–9.12；补充 S1.3、S3.3、S5.3–S5.4	同一运行全部片段、完整材料运输与已付库存的实际后继
§9	上述执行与模型底座	全环境残差恒等、原根可写请求、同债付款目的地、完整后缀、同次回答／回执／整账／后继
§10–11	定理 6.7 的两个层次	无总律付款良基与使用逐当前总律的有限结清分别证明；后继同债检查
§12	定理 8.4、8.6–8.7；命题 8.9	完整旧账／新增行同首写处置、丰富修订准确拆分
§13	原单位算术根与完整表示接口	素数三非零坐标、行列式湮灭、全部有限复观察无法恢复原整数消费者
§14	原执行器、配对语言、定理 9.12 的未来核	有限窗口四迹、字面纠正两迹、嵌套同次写入消费、真实种子与全部未来

第 9、14 节给出新原根消费与长计算的完整证明；补充材料展开第 5、7、8 节的 Type 库存、费用、准入和共尾接口。

附录 B　合同与计量单位

对象	必须消费的原对象
忠实终账	完整事件呈现与独立的普通出边穷尽证明
逐行处置	同一次整账编译、生成片段及确切余项
同债付款	原实际步骤、唯一同债目标及严格预算扣减
有限结清历史	付款良基性，加逐当前发射准确结清或实际付款的总律
最终结清	原终回执，加实际后继完整同债纤维为空
语法完成	原 AST 的全部节点、完整已付迹及常量端点
完成回答与续行	原完成动作、其回答与回执、同次完整账写入和规范后继
新责任	自己的实际来源、出生首步和独立主张分支

语法费以原执行节点为单位；同债预算从原开放见证读取；数学完成、同债付款和领域终止各使用自己的原收据。它们在同一实际链中相容，却没有被换成同一个数值量。依赖类型的“不可伪造”按第 1 节定义，实际终止与预算单位按相应源合同读取。

代码与数据可得性

WA1–WA40 的完整形式化证明、直接消费者、资源与复现程序绑定 H0mework 提交 `4301fa8c5bb19bb1e8c3824d7c69be406264ed12`。逐项映射登记各自原源 epoch、生产与消费者 owner、完整声明类型及验收回执；实际材料固定于 `f607eb646d4e9cddba3512755365842895af44ca`。13 个历史证明包分别保留独立 Lean 环境及准确入口，其中 WA38–WA40 使用 `H0mework.Papers.WholeLedgerAccountingRba7e274b`。第 11.7 节第三阶段付款实例使用 Y1 的证明修复，命题不变；其他 Y 材料保原 epoch。

复现指南固定 Lean 4.33.0 与 Mathlib v4.33.0。仓库根目录运行 `make check-whole-ledger-entry` 恢复完整源码与资源选集；以 `make bootstrap` 准备依赖，再运行 `make check-whole-ledger-full` 核查完整 import 构建、metadata trust 与完整类型；其实际结果由验收记录绑定。完整书面证明随正文与补充提供；共享过程结果采用过程核心第二版 DOI 10.5281/zenodo.23297175。图件随附可编辑矢量源与生成器。原创代码许可 Apache-2.0；论文及原创图件采用 CC BY 4.0。

参考文献

[1] David Ellerman. *On Double-Entry Bookkeeping: The Mathematical Treatment*. Accounting Education, 23(5), 483–501, 2014. DOI: `10.1080/09639284.2014.949803`. 作者预印本.

[2] George C. Necula and Peter Lee. *Safe Kernel Extensions Without Run-Time Checking*. Proceedings of OSDI, 229–243, 1996. DOI: `10.1145/238721.238781`. USENIX 原文.

[3] John C. Reynolds. *Separation Logic: A Logic for Shared Mutable Data Structures*. Proceedings of LICS, 55–74, 2002. 作者原文.

[4] Robert Atkey. *Amortised Resource Analysis with Separation Logic*. Logical Methods in Computer Science, 7(2:17), 2011. DOI: `10.2168/LMCS-7(2:17)2011`. 作者原文.

[5] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. Proceedings of PODS, 2007. 作者原文.

[6] 高健（Jian Gao）. 《状态不是历史，源才是：过程同一性、责任承接与最小修订》／ *The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes*. 第二版，2026. DOI: 10.5281/zenodo.23297175. 正文与完整证明补充。